

DSA: Trees (Binary Tree, BFS, BST) - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత Binary Tree, BFS (level-order), మరియు BST problems మళ్ళీ జీవితంలో మర్చిపోకూడదు. ప్రతి problem కి – ఎలా ఆలోచించాలి (intuition first), ఒక vivid real-life analogy, "ఈ subtree నుండి నాకు ఏమి కావాలి?" అనే recursive thinking, naive నుండి optimal వరకు thought process, clean commented JavaScript solution, పెన్సిల్‌తో గీసినట్టు dry run, complexity reasoning (recursion stack / tree height తో సహా), pattern takeaway, మరియు edge cases (null root, single node, skewed tree) – అన్నీ ఉంటాయి.

లక్ష్యం: DSA అస్సలు తెలియని person ని – ఎవరికైతే "tree" అంటే ఏమిటో, "recursion" అంటే ఏమిటో కూడా సరిగ్గా తెలియదే – వాళ్ళని SSE (Senior Software Engineer) interview లో confident గా ఈ 21 problems solve చేసేలా తయారు చేయడం. మనం facts బట్టి పట్టడం కాదు – **ఎలా ఆలోచించాలో** నేర్చుకుంటాం. Tree అంటే recursion; recursion అంటే "నా పని చిన్నదిగా చేసి, మిగతాది children కి అప్పజెప్పడం". ఈ ఒక్క idea పట్టుకుంటే, ఈ 21 మాత్రమే కాదు, వీటిలాంటి 200 tree problems కూడా solve చేయగలవు.

గమనిక: Big-O notation, recursion అంటే ఎలా పని చేస్తుంది, call stack, queue, hash map basics, time/space complexity – ఇలాంటి పునాదులు (fundamentals) [DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. అవి ముందు చదివితే ఈ guide ఇంకా సులభంగా అర్థమవుతుంది. ఇక్కడ మనం నేరుగా Tree patterns లోకి దిగుతాం.

విషయ సూచిక (Table of Contents)

Pattern Primers (ముందు వీటిని చదువు – problems కి foundation)

- Pattern: Tree DFS (recursion – "subtree నుండి ఏమి return చేయాలి", preorder/inorder/postorder, ఎప్పుడు ఏది)
- Pattern: Tree BFS (level-order via queue, level-by-level template) – *problem 15 ముందు*
- Pattern: BST property (inorder = sorted, bounds తో search/validate) – *problem 19 ముందు*

Part 1 – Binary Tree (General DFS)

1. Maximum Depth of Binary Tree (LeetCode #104) – Easy
2. Same Tree (LeetCode #100) – Easy
3. Invert Binary Tree (LeetCode #226) – Easy
4. Symmetric Tree (LeetCode #101) – Easy
5. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode #105) – Medium
6. Construct Binary Tree from Inorder and Postorder Traversal (LeetCode #106) – Medium
7. Populating Next Right Pointers in Each Node II (LeetCode #117) – Medium
8. Flatten Binary Tree to Linked List (LeetCode #114) – Medium
9. Path Sum (LeetCode #112) – Easy

10. Sum Root to Leaf Numbers (LeetCode #129) — Medium
11. Binary Tree Maximum Path Sum (LeetCode #124) — Hard
12. Binary Search Tree Iterator (LeetCode #173) — Medium
13. Count Complete Tree Nodes (LeetCode #222) — Easy
14. Lowest Common Ancestor of a Binary Tree (LeetCode #236) — Medium

Part 2 — Binary Tree BFS (Level Order)

15. Binary Tree Right Side View (LeetCode #199) — Medium
16. Average of Levels in Binary Tree (LeetCode #637) — Easy
17. Binary Tree Level Order Traversal (LeetCode #102) — Medium
18. Binary Tree Zigzag Level Order Traversal (LeetCode #103) — Medium

Part 3 — Binary Search Tree (BST)

19. Minimum Absolute Difference in BST (LeetCode #530) — Easy
20. Kth Smallest Element in a BST (LeetCode #230) — Medium
21. Validate Binary Search Tree (LeetCode #98) — Medium

Pattern: Tree DFS

వివరణ

Tree అంటే ఒక data structure — ఒక **root** (మూలం) node ఉంటుంది, దానికి children ఉంటాయి, ఆ children కి మళ్ళీ children... ఇలా కొమ్మలు విస్తరిస్తాయి. **Binary Tree** అంటే ప్రతి node కి **ఎక్కువలో ఎక్కువ 2 children** — **left** మరియు **right**. Child లేకపోతే ఆ pointer **null**. Children లేని node ని **leaf** అంటారు.

ఒక node ని ఇలా represent చేస్తాం (ఈ guide అంతటా **ఇదే** TreeNode వాడతాం):

```
class TreeNode {
    constructor(val = 0, left = null, right = null) {
        this.val = val; // ఈ node లో ఉన్న value
        this.left = left; // ఎడమ child (లేదా null)
        this.right = right; // కుడి child (లేదా null)
    }
}
```

ఇక్కడ కీలకమైన మాట: **ప్రతి node ఒక subtree కి root**. **root.left** అనేది కేవలం ఒక node కాదు — అది ఒక **మొత్తం చెట్టు** (left subtree). ఇదే tree recursion కి అత్త. చెట్టుని చూసినప్పుడు "ఇది root, దీనికి రెండు చిన్న చెట్లు వేలాడుతున్నాయి" అని ఆలోచించు — ఆ రెండు చిన్న చెట్లు కూడా సరిగ్గా ఇలాంటివే, కేవలం చిన్నవి.

DFS (Depth-First Search) అంటే — ఒక కొమ్మని **చివరిదాకా (లోతుకి)** వెళ్ళి, తర్వాత వెనక్కి వచ్చి ఇంకో కొమ్మ చూడటం. Tree లో DFS అంటే almost ఎప్పుడూ **recursion**. Recursion mindset ఒక్క వాక్యంలో:

"ఈ node దగ్గర నా చిన్న పని ఏమిటి? మిగతా పని (left subtree, right subtree) కి ఒక function ని మళ్ళీ పిలిచి, వాళ్ళ answers తీసుకుని, నా answer కలిపి return చేస్తా."

ప్రతి recursive function కి రెండు భాగాలు:

1. **Base case** — ఆగే చోటు. Tree లో ఇది almost ఎప్పుడూ `if (node == null) return ...; .` ఖాళీ చెట్టుకి answer ఏమిటో చెప్పు (depth 0, sum 0, true, null — problem బట్టి).
2. **Recursive case** — `node.left` , `node.right` కి function ని పిలిచి, వాళ్ళ results ని **combine** చేసి return చెయ్యి.

మూడు DFS orders (ఎప్పుడు node ని "process" చేస్తామో అనేదే తేడా):

Order	ఏ క్రమంలో	ఎప్పుడు వాడతాం
Preorder (root → left → right)	ముందు node, తర్వాత children	Tree ని copy/serialize చేయడం, top-down గా information పంపడం (parent → child)
Inorder (left → root → right)	ఎడమ subtree పూర్తయ్యాక node, తర్వాత కుడి	BST లో — inorder ఇస్తుంది sorted sequence! (చాలా కీలకం)
Postorder (left → right → root)	ముందు children ఇద్దరూ, తర్వాత node	node కి children నుండి information కావాలి అప్పుడు (height, sum, "ఈ subtree valid ఆ?") — bottom-up

గుర్తుంచుకోవడానికి: "**root ఎప్పుడు?**" — Pre = మొదట, In = మధ్యలో, Post = చివర. చాలా tree problems **postorder** (children నుండి answer తెచ్చుకుని node దగ్గర combine) పద్ధతిలో పరిష్కారమవుతాయి.

Real-life Scenario

ఒక **company org chart** ని ఊహించుకో. CEO (root) కింద ఇద్దరు VPs (left, right). ప్రతి VP కింద మళ్ళీ managers, వాళ్ళ కింద engineers (leaves). ఇప్పుడు CEO కి "మొత్తం company లో ఎంత మంది ఉన్నారు?" తెలియాలి అనుకో. CEO ప్రతి ఒక్కరినీ లెక్కపెట్టడు — ఆయన కేవలం **ఇద్దరు VPs ని అడుగుతాడు**: "మీ కింద ఎంతమంది?" ప్రతి VP అదే పని చేస్తాడు — తన managers ని అడుగుతాడు. చివరికి engineer (leaf) కి కింద ఎవరూ లేరు కాబట్టి "నేను ఒక్కడినే (1)" అంటాడు. Numbers పైకి వెనక్కి వస్తూ కలుస్తాయి. CEO చేసిన పని: **1** (నేను) + VP1 చెప్పింది + VP2 చెప్పింది . ఇదే **postorder recursion** — పని కింద అప్పజెప్పి, children answers కలపడం.

DFS "depth first" అంటే ఏమిటి? CEO ముందు VP1 branch **మొత్తం** (అట్టడుగు engineer దాకా) పూర్తిగా అన్వేషించి, ఆ తర్వాతే VP2 branch చూస్తాడు. ఒక దారిలో చివరిదాకా వెళ్ళి, తర్వాత వెనక్కి — అదే depth-first.

ఎలా గుర్తించాలి (Recognition Signals)

ఈ signals కనిపిస్తే Tree DFS (recursion) గురించి ఆలోచించు:

- **Input ఒక TreeNode root** — దాదాపు అన్ని tree problems recursion తో మొదలవుతాయి.
- "**depth / height / path / root-to-leaf / ancestor / subtree**" అనే మాటలు.
- "**ప్రతి node కి, దాని children మీద ఆధారపడి ఏదో లెక్కించాలి**" — classic postorder.
- **రెండు subtrees ని compare/combine చేయాలి** — same tree, symmetric, LCA.

- **BST + sorted / kth / range** అనే మాట → inorder (శీద BST primer).

Template Code (JavaScript)

```
// మూడు traversals - recursion తో. n = nodes సంఖ్య.
function preorder(node, out = []) {
  if (node === null) return out; // base case
  out.push(node.val);           // 1) root ని process (ముందు)
  preorder(node.left, out);      // 2) ఎడమ subtree
  preorder(node.right, out);     // 3) కుడి subtree
  return out;
}

function inorder(node, out = []) {
  if (node === null) return out;
  inorder(node.left, out);       // 1) ఎడమ
  out.push(node.val);            // 2) root ని process (మధ్యలో)
  inorder(node.right, out);      // 3) కుడి
  return out;
}

function postorder(node, out = []) {
  if (node === null) return out;
  postorder(node.left, out);     // 1) ఎడమ
  postorder(node.right, out);    // 2) కుడి
  out.push(node.val);            // 3) root ని process (చివర)
  return out;
}

// --- "children నుండి answer తెచ్చుకునే" postorder skeleton (చాలా వాడతాం) ---
function solve(node) {
  if (node === null) return /* ఖాళీ చెట్టు answer, ఉదా: 0 / true / null */;
  const leftAns = solve(node.left); // ఎడమ subtree నన్ను ఏమి చెప్తుంది?
  const rightAns = solve(node.right); // కుడి subtree నన్ను ఏమి చెప్తుంది?
  // ఇప్పుడు నా node value + leftAns + rightAns కలిపి NA answer తయారు చేసి return
  return /* combine(node.val, leftAns, rightAns) */;
}
```

Complexity

- **Time: $O(n)$** – ప్రతి node ని ఎక్కువలో ఎక్కువ ఒకసారి visit చేస్తాం (n = మొత్తం nodes).
- **Space: $O(h)$** – recursion **call stack** లోతు = tree **height h** . చెట్టు **balanced** అయితే $h \approx \log n$ (బాగుంది). చెట్టు **skewed** (ఒక వైపు వేలాడే గొలుసులా) అయితే $h = n$ (worst case, $O(n)$ stack). ఈ height ఆలోచనే tree space complexity కి కీలకం – ఇంటర్వ్యూలో "space?" అంటే " $O(h)$, worst case $O(n)$ " అని చెప్పు.

1. Maximum Depth of Binary Tree (LeetCode #104) – Easy

సమస్య: ఒక binary tree root ఇస్తారు. దాని **maximum depth** (గరిష్ట లోతు) return చెయ్యాలి. Maximum depth అంటే – root నుండి **అత్యంత దూరపు leaf** వరకు ఉన్న path లో **nodes సంఖ్య**.

Constraints: nodes సంఖ్య $[0, 10^4]$; $-100 \leq \text{Node.val} \leq 100$. (root null అవ్వచ్చు – ఖాళీ చెట్టు.)

ఉదాహరణ:



Input: root = [3,9,20,null,null,15,7]

Output: 3 (దారి 3 → 20 → 15 లేదా 3 → 20 → 7, పొడవు 3 nodes)

ఎలా ఆలోచించాలి:

"మొత్తం చెట్టు depth" అనేది పెద్ద ప్రశ్నలా అనిపిస్తుంది. కానీ recursion trick ఇది: పెద్ద ప్రశ్నని, అదే ప్రశ్న యొక్క చిన్న version లతో చెప్పు.

ఒక node దగ్గర నిలబడి ఆలోచించు. నా depth ఏమిటి? = 1 (నేను) + నా కింద ఉన్న అత్యంత లోతైన subtree depth. అంటే నాకు left subtree depth మరియు right subtree depth తెలిస్తే, నా answer = 1 + max(ఆ రెండు) . ఇద్దరు children ని "మీ లోతు ఎంత?" అని అడిగి, పెద్దది తీసుకుని, నాకోసం 1 కలుపుతా.

మరి ఆగేదెప్పుడు? ఖాళీ చెట్టు (null) దగ్గర – దాని depth 0 (nodes లేవు). ఇదే base case. ఇది postorder ఆలోచన: ముందు children లోతులు తెలియాలి, తర్వాత నా లోతు.

Optimal Approach:

Insight: $\text{depth}(\text{node}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, base case $\text{depth}(\text{null}) = 0$. ఇది clean postorder recursion – brute force అంటూ ప్రత్యేకంగా ఏమీ అవసరం లేదు, ఇదే optimal.

Plan:

1. root === null → return 0.
2. లేకపోతే left depth, right depth recursion తో తెచ్చుకో.
3. 1 + Math.max(left, right) return.

Solution (JavaScript):

```
function maxDepth(root) {
  // base case: ఖాళీ చెట్టు లోతు 0
  if (root === null) return 0;

  // ఇద్దరు children ని "మీ లోతు ఎంత?" అని అడుగు (recursion)
  const leftDepth = maxDepth(root.left);
  const rightDepth = maxDepth(root.right);

  // నా లోతు = 1 (నేను) + లోతైన child వైపు లోతు
  return 1 + Math.max(leftDepth, rightDepth);
}
```

Dry Run: root = [3,9,20,null,null,15,7]

```

maxDepth(3)
├─ maxDepth(9) // leaf 9
│   ├── maxDepth(null) = 0
│   ├── maxDepth(null) = 0
│   └─ return 1 + max(0,0) = 1
├─ maxDepth(20)
│   ├── maxDepth(15) = 1 // leaf
│   ├── maxDepth(7) = 1 // leaf
│   └─ return 1 + max(1,1) = 2
└─ return 1 + max(1, 2) = 3 ✓

```

లోతైన కొమ్మ (20 వైపు) 2 తెచ్చింది, దానికి root కోసం 1 కలిపి 3.

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి visit.
- **Space:** $O(h)$ – recursion stack, h = tree height. Balanced అయితే $O(\log n)$, skewed అయితే $O(n)$.

గుర్తుంచుకోవాల్సినది:

ఇది **అన్ని tree problems కి mother template**. "నా answer = నా node + children ల answers combine" అనే postorder నమూనా. ఇదే idea తో పరిష్కారమయ్యే బంధువులు: **Minimum Depth** (**min** వాడు, కానీ ఒక child null అయితే జాగ్రత్త – కింద చూడు), **Balanced Binary Tree** (#110), **Diameter of Binary Tree** (#543 – ప్రతి node దగ్గర **left+right** track చేస్తూ). చెట్టు depth = recursion యొక్క hello-world.

సాధారణ తప్పులు:

- **null base case మర్చిపోవడం** → `root.left` మీద `null.left` చదవబోయి crash (Cannot **read** properties of **null**). Recursion లో మొదటి line ఎప్పుడూ null check.
- **Minimum Depth తో గందరగోళం:** Max depth కి **max** సరిపోతుంది. కానీ **minimum** depth కి plain **1 + min(left, right)** తప్పు – ఒక child null అయితే (ఉదా: node కి కుడి child మాత్రమే ఉంది), null వైపు 0 వచ్చి తప్పుగా చిన్న answer వస్తుంది. అక్కడ leaf కి మాత్రమే min తీసుకోవాలి. ఈ problem కి max కాబట్టి safe, కానీ తేడా తెలుసుకో.
- **Depth vs Height గందరగోళం:** ఇక్కడ "depth" = root నుండి nodes count. కొందరు edges లెక్కిస్తారు (అప్పుడు single node depth 0). LeetCode ఇక్కడ **nodes count** వాడుతుంది (single node → 1).

2. Same Tree (LeetCode #100) – Easy

సమస్య: రెండు binary trees **p** మరియు **q** ఇస్తారు. అవి **ఒకేలా (identical)** ఉన్నాయా చెప్పాలి. ఒకేలా అంటే – **structure ఒకటే** (అదే చోట్ల nodes, అదే చోట్ల null) మరియు ప్రతి corresponding node లో **అదే value**.

Constraints: రెండు trees లో nodes `[0, 100]` ; `-10^4 <= Node.val <= 10^4` .

ఉదాహరణ:

```

p:  1      q:  1      → true (structure + values match)
   / \    / \
  2   3  2   3

```

p: 1 q: 1 → false (structure తేడా – 2 ఒకవేట ఎడమ, ఇంకోవేట కుడి)

```

  /      \
 2        2

```

ఎలా ఆలోచించాలి:

రెండు చెట్లు ఒకటేనా అని ఎలా చెప్తావు? ఒకేసారి **రెండింటి roots దగ్గర నిలబడి** compare చెయ్యి. మూడు ప్రశ్నలు:

1. ఇద్దరూ null ఆ? → అయితే ఈ చీట ఖాళీ, match (**true**).
2. ఒక్కడే null ఆ? → ఒకరికి node ఉంది, ఇంకొకరికి లేదు → structure తేడా (**false**).
3. ఇద్దరికీ node ఉంది కానీ **values తేడా** ఆ? → **false** .

ఈ మూడూ దాటితే, ఈ node OK. ఇప్పుడు అదే ప్రశ్నని **ఎడమ subtrees జతకి** మరియు **కుడి subtrees జతకి** అడుగు. రెండూ **true** అయితేనే మొత్తం **true** . ఇది "రెండు చెట్లని parallel గా నడిపే" recursion.

Optimal Approach:

Insight: రెండు nodes ని ఏకకాలంలో traverse చేస్తూ compare. **p** నుండి ఎడమకి వెళ్తే **q** నుండి ఎడమకే వెళ్ళాలి (mirror కాదు – exact match). Base cases రెండు null-గోళాలు.

Solution (JavaScript):

```

function isSameTree(p, q) {
  // ఇద్దరూ null → ఈ చీట ఖాళీ, match
  if (p === null && q === null) return true;
  // ఒక్కడే null → structure తేడా
  if (p === null || q === null) return false;
  // ఇద్దరికీ node ఉంది కానీ value తేడా → కాదు
  if (p.val !== q.val) return false;

  // ఈ node OK. ఇప్పుడు ఎడమ-ఎడమ, కుడి-కుడి subtrees కూడా match అవ్వాలి
  return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}

```

Dry Run: p = [1,2,3] , q = [1,2,3]

```

isSameTree(1,1): null? కాదు. 1===1 ✓ → left & right చూడు
├─ isSameTree(2,2): 2===2 ✓ → left(null,null)=true, right(null,null)=true → true
└─ isSameTree(3,3): 3===3 ✓ → true
return true && true = true ✓

```

Mismatch case p=[1,2] , q=[1,null,2] :

```

isSameTree(1,1) ✓ → left: isSameTree(2, null) → ఒక్కడే null → false
మొత్తం: false && ... = false ✓ ( && short-circuit – కుడి చూడనే అక్కర్లేదు )

```

Complexity:

- **Time:** $O(n)$ – n = చిన్న చెట్టు nodes (mismatch వస్తే ముందే ఆగుతాం, worst case రెండూ సమానం → $O(n)$).

- **Space:** $O(h)$ — recursion stack, h = height.

గుర్తుంచుకోవాల్సినది:

రెండు nodes ని ఏకకాలంలో compare చేసే pattern. Base cases క్రమం కీలకం: ముందు "ఇద్దరూ null?", తర్వాత "ఒక్కడే null?", తర్వాత value. ఈ నమూనా వాడే బంధువులు: **Symmetric Tree** (#101 — తర్వాతి problem, కేవలం mirror గా compare), **Subtree of Another Tree** (#572 — ప్రతి node దగ్గర isSameTree పీలుపు), **Merge Two Binary Trees**.

సాధారణ తప్పులు:

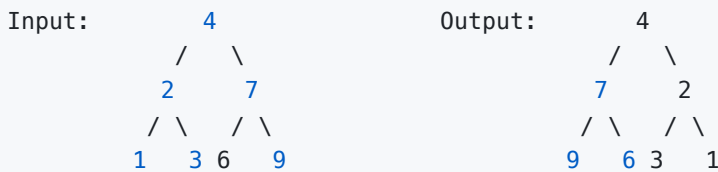
- **Base cases క్రమం తప్పడం:** "ఒక్కడే null" check ముందు value compare చేస్తే, null.val చదవబోయి crash. ముందు null cases handle చెయ్యి.
- **== vs === :** JS లో === వాడు. Values numbers కాబట్టి పెద్ద తేడా లేదు, కానీ అలవాటు మంచిది.
- **Mirror తో గందరగోళం:** Same tree = left↔left, right↔right. Symmetric (mirror) = left↔right. ఈ రెండింటినీ కలిపేయకు.

3. Invert Binary Tree (LeetCode #226) — Easy

సమస్య: ఒక binary tree root ఇస్తారు. దాన్ని **invert (mirror)** చేసి return చెయ్యాలి — అంటే ప్రతి node దగ్గర **left, right children ని పరస్పరం మార్చడం** (అద్దంలో చూసినట్లు).

Constraints: nodes $[0, 100]$; $-100 \leq \text{Node.val} \leq 100$.

ఉదాహరణ:



గమ్యత్తు: ఇదే problem వల్ల ఒక famous tweet — "Google interview reject me because I couldn't invert a binary tree on a whiteboard." So ఇది తెలిసి ఉండటం SDE culture లో ఒక rite of passage!

ఎలా ఆలోచించాలి:

"మొత్తం చెట్టుని mirror చెయ్యి" — పెద్దగా అనిపిస్తుంది. కానీ ఒక node దగ్గర ఆలోచించు: mirror అంటే **నా left, right ని swap చేయడం**. కానీ కేవలం top level swap సరిపోదు — కింద ఉన్న ప్రతి subtree కూడా mirror అవ్వాలి.

Recursion: "నా left subtree ని పూర్తిగా invert చెయ్యి, నా right subtree ని పూర్తిగా invert చెయ్యి, తర్వాత ఆ రెండింటినీ swap చేసి నా కింద తగిలించు." ఇద్దరు children తమ పని పూర్తి చేసుకుని invert అయిన subtrees return చేస్తారు, నేను వాటిని cross చేసి పెడతా. Base case: null → invert చేయడానికి ఏమీ లేదు, null return.

Optimal Approach:

Insight: `invert(node)` = children ని invert చేసి, swap చేసి return. Preorder (ముందు swap, తర్వాత recurse) లేదా postorder (ముందు recurse, తర్వాత swap) — రెండూ పని చేస్తాయి. కింద postorder-ish (recurse then swap)

రాస్టున్నా – clean.

Solution (JavaScript):

```
function invertTree(root) {  
  if (root === null) return null;          // base case: ఖాళీ → ఏమీ లేదు  
  
  // ఇద్దరు children ని పూర్తిగా invert చేయించు (recursion)  
  const invertedLeft = invertTree(root.left);  
  const invertedRight = invertTree(root.right);  
  
  // ఇప్పుడు swap: పాత left స్థానంలో right, పాత right స్థానంలో left  
  root.left = invertedRight;  
  root.right = invertedLeft;  
  
  return root;  
}
```

Shortcut: పైన invertedLeft/Right పేర్లు clarity కోసం. Interview లో ఒక్క line లో: `[root.left, root.right] = [invertTree(root.right), invertTree(root.left)];` – destructuring swap. కానీ మొదట స్పష్టంగా రాయడం మంచిది.

Dry Run: root = [4, 2, 7, 1, 3, 6, 9]

```
invert(4)  
├ invert(2) → children 1,3 swap → 2 becomes (left=3, right=1)  
├ invert(7) → children 6,9 swap → 7 becomes (left=9, right=6)  
└ swap 2 & 7 → 4 becomes (left=7, right=2)
```

ఫలితం:

```
      4  
     /\   
    7  2  
   /\ /\   
  9 6 3 1  ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి swap.
- **Space:** $O(h)$ – recursion stack.

గుర్తుంచుకోవాల్సినది:

"Mirror / swap children / reverse structure" అంటే ఈ pattern. Node దగ్గర **children ని modify** చేసే tree problems (in-place structural change) కి నమూనా. దీన్ని **BFS/queue తో కూడా** చేయవచ్చు (ప్రతి node pop చేసి left/right swap చేసి children ని push). Recursion elegant గా ఉంటుంది.

సాధారణ తప్పులు:

- **Swap ముందే recurse అయిపోవడం (order bug):** పైన నేను children ని ముందు invert చేసి తర్వాత swap చేశా – safe. కానీ ఒకవేళ preorder రాస్తే (ముందు swap): `let t = root.left; root.left = root.right;`

root.right = t; చేసి తర్వాత invertTree(root.left) పిలవాలి. పొరపాటున let t = root.left; root.left = invertTree(root.right); root.right = invertTree(t); – ఇక్కడ t పాత left ని పట్టుకుంది కాబట్టి OK, కానీ swap మధ్యలో recurse చేస్తే original pointers గల్లంతవుతాయి. క్రమం జాగ్రత్తగా.

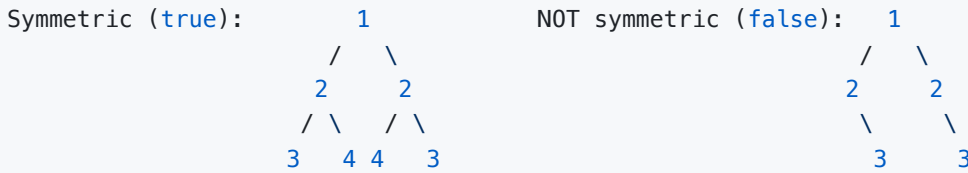
- **null return మర్చిపోవడం:** base case లో return null (లేదా function చివర return root) మర్చిపోతే undefined వచ్చి పైన tie చేసేటప్పుడు tree విరిగిపోతుంది.
- **return root మర్చిపోవడం:** invert చేసి root return చేయకపోతే caller కి tree అందదు.

4. Symmetric Tree (LeetCode #101) – Easy

సమస్య: ఒక binary tree root ఇస్తారు. అది తనచుట్టూ తానే mirror-symmetric ఆ (అంటే మధ్య నిలువు గీత గీస్తే ఎడమ సగం, కుడి సగం అద్దంలో ఉన్నట్టు) చెప్పాలి.

Constraints: nodes [1, 1000] ; -100 <= Node.val <= 100 .

ఉదాహరణ:



Left subtree, right subtree ఒకదానికొకటి mirror అయితే symmetric.

ఎలా ఆలోచించాలి:

"చెట్టు తనచుట్టూ mirror ఆ?" – దీన్ని విడగొడితే: root యొక్క left subtree, root యొక్క right subtree ఒకదానికొకటి mirror ఆ? అనే ప్రశ్న.

రెండు subtrees mirror అవ్వాలంటే ఏమి కావాలి? ఇది Same Tree (#2) లాంటిదే, కానీ ఒక్క twist తో: mirror లో ఎడమ యొక్క ఎడమ = కుడి యొక్క కుడి కాదు – ఎడమ యొక్క ఎడమ = కుడి యొక్క కుడి? కాదు! అద్దంలో దిశలు తిరుగుతాయి: A యొక్క left ↔ B యొక్క right, A యొక్క right ↔ B యొక్క left.

కాబట్టి రెండు nodes a , b mirror అవ్వాలంటే: (1) a.val === b.val , (2) a.left mirror b.right , (3) a.right mirror b.left . Same Tree లో straight (left-left, right-right) పోల్చాం; ఇక్కడ cross (left-right, right-left) పోల్చుతాం. అదే తేడా.

Optimal Approach:

Insight: ఒక helper isMirror(a, b) రాయ – రెండు subtrees mirror ఆ చెప్పేది. root కి isMirror(root.left, root.right) పిలువు. Cross comparison గుర్తుంచుకో.

Solution (JavaScript):

```

function isSymmetric(root) {
  if (root === null) return true; // ఖాళీ చెట్టు symmetric

  // రెండు subtrees ఒకదానికొకటి mirror ఆ?

```

```
function isMirror(a, b) {
  if (a === null && b === null) return true; // ఇద్దరూ ఖాళీ → mirror OK
  if (a === null || b === null) return false; // ఒక్కడే ఖాళీ → కాదు
  // value match + CROSS comparison (a.left↔b.right, a.right↔b.left)
  return a.val === b.val
    && isMirror(a.left, b.right)
    && isMirror(a.right, b.left);
}

return isMirror(root.left, root.right);
}
```

Dry Run: root = [1,2,2,3,4,4,3]

```
isMirror(2L, 2R): 2===2 ✓
├ isMirror(3, 3): a.left=3 ↔ b.right=3 → 3===3 ✓, children null → true
└ isMirror(4, 4): 4===4 ✓ → true
return true ✓ → tree symmetric
```

NOT symmetric [1,2,2,null,3,null,3] :

```
isMirror(2L, 2R): 2===2 ✓
├ isMirror(2L.left=null, 2R.right=3): ఒక్కడే null → false
return false ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి.
- **Space:** $O(h)$ – recursion stack.

గుర్తుంచుకోవాల్సినది:

Symmetric = **cross comparison** తో **Same Tree**. ఈ "రెండు nodes ని mirror గా నడపడం" pattern గుర్తుపెట్టుకో. Iterative గా queue తో కూడా చేయవచ్చు – జతలుగా (a, b) push చేస్తూ, ప్రతి జతకి a.left↔b.right, a.right↔b.left push. Key insight: **directions reverse** (left↔right).

సాధారణ తప్పులు:

- **Straight comparison వాడటం (అతిపెద్ద తప్పు):** `isMirror(a.left, b.left)` రాస్తే అది Same Tree అవుతుంది, symmetric కాదు. Cross (`a.left, b.right`) తప్పనిసరి.
- **root.val ని compare చేయడం:** root ఒక్కటే – దాన్ని ఎవరితో compare? Root ని పోల్చకుండా దాని **రెండు children subtrees** ని పోల్చాలి.
- **Single node:** [1] → left, right రెండూ null → `isMirror(null,null)=true` → symmetric. సరిగ్గా handle అవుతుంది.

5. Construct Binary Tree from Preorder and Inorder Traversal (LeetCode #105) — Medium

సమస్య: ఒక binary tree యొక్క **preorder** traversal (preorder[]) మరియు **inorder** traversal (inorder[]) arrays ఇస్తారు. అసలు చెట్టుని **తిరిగి నిర్మించి (construct)** return చెయ్యాలి.

Constraints: $1 \leq n \leq 3000$; values **unique** (డూప్లికేట్లు లేవు — ఇది కీలకం); ఇచ్చిన arrays నిజంగా ఒక valid tree వి.

ఉదాహరణ:

```
preorder = [3,9,20,15,7]
inorder = [9,3,15,20,7]
```

Output:

```
      3
     / \
    9  20
     \  / \
      15 7
```

ఎలా ఆలోచించాలి:

రెండు traversals ఇచ్చి "చెట్టు కట్టు" అంటే మొదట భయం. కానీ ప్రతి traversal ఏమి చెప్తుందో గుర్తుచేసుకో:

- **Preorder** = [root, ...ఎడమ subtree..., ...కుడి subtree...]. కాబట్టి **preorder మొదటి element ఎప్పుడూ root!** (ఇక్కడ 3).
- **Inorder** = [...ఎడమ subtree..., root, ...కుడి subtree...]. కాబట్టి inorder లో root (3) ని కనిపెడితే, **దాని ఎడమవైపు అంతా left subtree, కుడివైపు అంతా right subtree!**

అదే insight. Preorder నుండి root తీసుకో → inorder లో దాని స్థానం చూడు → అది inorder ని రెండు ముక్కలుగా (ఎడమ subtree elements, కుడి subtree elements) విడగొడుతుంది. ఇప్పుడు అదే process ని ఆ రెండు ముక్కలకి recursion గా apply చెయ్యి. చెట్టు దానంతట అదే తయారవుతుంది.

Preorder ని ఎడమ నుండి కుడికి **వరుసగా consume** చేస్తాం (ఒక pointer pre). ఎందుకంటే preorder = root, తర్వాత మొత్తం left subtree (preorder లోనే), తర్వాత right subtree. కాబట్టి root తీశాక, తర్వాతి preorder element = left subtree యొక్క root. అందుకే **left ని ముందు build** చేస్తాం.

Brute Force / naive:

ప్రతి recursion లో inorder లో root ని **indexOf** తో వెతికితే $O(n)$ అవుతుంది, subtrees కి slice() చేస్తే మరో $O(n)$ copy. మొత్తం $O(n^2)$ time + $O(n^2)$ space (slices). చిన్న inputs కి పని చేస్తుంది కానీ $n=3000$ కి waste.

- **ఎందుకు సరిపోదు:** పదేపదే linear search + array copy. Inorder positions ఒక **hash map** లో ముందే store చేస్తే lookup $O(1)$; slice బదులు **index ranges** (l, r) పంపితే copy అక్కర్లేదు → $O(n)$.

Optimal Approach:

Insight: (1) preorder positions ని ఎడమ నుండి కుడికి pointer తో consume, (2) inorder value→index ని hash map లో precompute ($O(1)$ lookup), (3) slice కాకుండా **inorder range [inLeft, inRight]** పంపు.

Plan:

1. idx map: inorder value $\rightarrow$ index.
2. pre = 0 pointer (preorder లో ఎక్కడ ఉన్నామో).
3. build(inLeft, inRight) : range ఖాళీ (inLeft > inRight) $\rightarrow$ null. లేకపోతే rootVal = preorder[pre++] , inorder లో దాని mid index తీసుకో, ఎడమ subtree ముందు (build(inLeft, mid-1)), తర్వాత కుడి (build(mid+1, inRight)).

Solution (JavaScript):

```
function buildTree(preorder, inorder) {
  // inorder లో ప్రతి value ఎక్కడ ఉందో 0(1) లో తెలుసుకోవడానికి map
  const idx = new Map();
  for (let i = 0; i < inorder.length; i++) idx.set(inorder[i], i);

  let pre = 0; // preorder ని ఎడమ నుండి కుడికి వరుసగా consume చేస్తాం

  function build(inLeft, inRight) {
    if (inLeft > inRight) return null; // ఖాళీ range  $\rightarrow$  subtree లేదు

    const rootVal = preorder[pre++]; // preorder తదుపరిది = ఈ subtree root
    const root = new TreeNode(rootVal);
    const mid = idx.get(rootVal); // inorder లో root స్థానం

    // preorder క్రమం: root తర్వాత మొత్తం LEFT subtree  $\rightarrow$  left ని ముందు build
    root.left = build(inLeft, mid - 1); // inorder లో root కి ఎడమవైపు
    root.right = build(mid + 1, inRight); // inorder లో root కి కుడివైపు
    return root;
  }

  return build(0, inorder.length - 1);
}
```

Dry Run: preorder=[3,9,20,15,7] , inorder=[9,3,15,20,7]

```
pre=0  $\rightarrow$  root=3, mid(inorder లో 3)=1
  build left = inorder[0..0] = [9]: pre=1  $\rightarrow$  root=9, mid=0, left=[], right=[]  $\rightarrow$  leaf 9
  build right = inorder[2..4] = [15,20,7]: pre=2  $\rightarrow$  root=20, mid=3
    left = inorder[2..2]=[15]: pre=3  $\rightarrow$  leaf 15
    right = inorder[4..4]=[7] : pre=4  $\rightarrow$  leaf 7
సంపూర్ణ: 3(left=9, right=20(left=15, right=7)) ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి తయారు; map lookup $O(1)$.
- **Space:** $O(n)$ – map + recursion stack ($O(h)$). map dominates $\rightarrow O(n)$.

గుర్తుంచుకోవాల్సినది:

"Preorder మొదటిది = root; inorder root స్థానం subtrees ని విడగొడుతుంది" – ఈ ఒక్క వాక్యం. Pointer + hashmap + range recursion అనే combo. బంధువు: **#106 (inorder + postorder)** – తర్వాతి problem, అక్కడ

postorder చివరిది root, కుడిని ముందు build. రెండింటికీ inorder అవసరం (values ని ఎడమ/కుడి గా విడగొట్టడానికి).

సాధారణ తప్పులు:

- **pre ని global గా share చేయకపోవడం:** pre ని closure variable గా (function వెలుపల) ఉంచాలి, ప్రతి call కి పెరగాలి. Parameter గా pass చేస్తే increments కలవవు → చెట్టు తప్పు.
- **Right ని ముందు build చేయడం:** Preorder కి **తప్పకుండా left ముందు**. Right ముందు build చేస్తే pre pointer తప్పు elements తీసుకుంటుంది.
- **Slice తో $O(n^2)$:** పని చేస్తుంది కానీ interview లో "ఎలా optimize?" అంటే range + map చెప్పాలి.
- **Duplicate values:** ఈ approach unique values మీద ఆధారపడుతుంది (map single index). Duplicates ఉంటే ఈ method పని చేయదు – constraints unique అని చెప్తాయి.

6. Construct Binary Tree from Inorder and Postorder Traversal (LeetCode #106) – Medium

సమస్య: ఒక binary tree యొక్క **inorder** (`inorder[]`) మరియు **postorder** (`postorder[]`) traversals ఇస్తారు. అసలు చెట్టుని construct చేసి return చెయ్యాలి.

Constraints: $1 \leq n \leq 3000$; values **unique**; arrays valid tree వి.

ఉదాహరణ:

```
inorder = [9,3,15,20,7]
postorder = [9,15,7,20,3]
```

Output:

```
      3
     / \
    9   20
      / \
     15  7
```

ఎలా ఆలోచించాలి:

#105 (preorder+inorder) బాగా అర్థమైతే ఇది దాదాపు అదే – ఒక్క twist. గుర్తు తెచ్చుకో:

- **Postorder** = [...ఎడమ subtree..., ...కుడి subtree..., root]. కాబట్టి **postorder చివరి element ఎప్పుడూ root!** (ఇక్కడ 3).
- **Inorder** = [...ఎడమ..., root, ...కుడి...] – #105 లోలాగే, inorder లో root స్థానం subtrees ని విడగొడుతుంది.

కాబట్టి postorder ని **కుడి నుండి ఎడమకి (వెనుక నుండి)** consume చేస్తాం (pointer post చివరి index నుండి తగ్గుతూ). Postorder ని రివర్స్ చేస్తే: [root, ...కుడి subtree..., ...ఎడమ subtree...]. అంటే root తీశాక **తర్వాతిది కుడి subtree యొక్క root!** కాబట్టి ఇక్కడ **కుడిని ముందు build** చేయాలి (preorder కి opposite).

ఒక్క వాక్యంలో తేడా: #105 preorder → root ముందు → **left ముందు build**. #106 postorder(వెనుక) → root చివర → **right ముందు build**.

Brute Force / naive: #105 లాగే – inorder లో indexOf + slice → $O(n^2)$. Map + range తో $O(n)$ కి తగ్గించవచ్చు.

Optimal Approach:

Insight: postorder ని చివర నుండి consume; inorder value→index map; range recursion; **right ని ముందు build.**

Solution (JavaScript):

```
function buildTree(inorder, postorder) {  
  const idx = new Map();  
  for (let i = 0; i < inorder.length; i++) idx.set(inorder[i], i);  
  
  let post = postorder.length - 1; // postorder ని చివరి నుండి వెనక్కి consume  
  
  function build(inLeft, inRight) {  
    if (inLeft > inRight) return null; // ఖాళీ range → subtree లేదు  
  
    const rootVal = postorder[post--]; // postorder చివరిది = ఈ subtree root  
    const root = new TreeNode(rootVal);  
    const mid = idx.get(rootVal); // inorder లో root స్థానం  
  
    // postorder ని వెనుక నుండి → root తర్వాత RIGHT subtree వస్తుంది → right ముందు build  
    root.right = build(mid + 1, inRight); // inorder లో root కి కుడివైపు  
    root.left = build(inLeft, mid - 1); // inorder లో root కి ఎడమవైపు  
    return root;  
  }  
  
  return build(0, inorder.length - 1);  
}
```

Dry Run: inorder=[9,3,15,20,7] , postorder=[9,15,7,20,3]

```
post=4 → root=3, mid(inorder లో 3)=1  
  build RIGHT = inorder[2..4]=[15,20,7]: post=3 → root=20, mid=3  
    right = inorder[4..4]=[7]: post=2 → leaf 7  
    left = inorder[2..2]=[15]: post=1 → leaf 15  
  build LEFT = inorder[0..0]=[9]: post=0 → leaf 9  
సంపూర్ణ: 3(left=9, right=20(left=15, right=7)) ✓
```

గమనించు: post pointer 4→3→2→1→0 క్రమంలో values 3,20,7,15,9 తీసుకుంది — postorder ని రివర్స్ చేస్తే root, right, left క్రమం.

Complexity:

- **Time:** $O(n)$. **Space:** $O(n)$ (map + recursion stack).

గుర్తుంచుకోవాల్సినది:

#105, #106 జంట. **Inorder ఎప్పుడూ కావాలి** (subtrees విడగొట్టడానికి). రెండో array (pre/post) root ఎక్కడో చెప్తుంది:

pre → మొదట (left build first), post → చివర (right build first). ఈ ఒక్క మెలిక పట్టుకుంటే రెండూ ఒకటే.

(Preorder + Postorder మాత్రం **unique** tree ఇవ్వవు — inorder లేకపోతే ambiguous.)

సాధారణ తప్పులు:

- **Left ని ముందు build చేయడం (#105 అలవాటుతో):** ఇక్కడ postorder-backward కాబట్టి **right ముందు**. తప్పు క్రమం → post pointer తప్పు root లు తీసి చెట్టు తలకిందులు.
- **post pointer దిశ:** చివరి index నుండి **తగ్గుతూ** (`post--`). పెంచితే తప్పు.
- **build calls క్రమం vs assignment క్రమం:** `root.right`, `root.left` ఏ ఆర్డర్లో అయినా *assign* చేయవచ్చు, కానీ **build() calls మాత్రం right ముందు execute** అవ్వాలి (post-- side effect క్రమం కీలకం). పైన right line ముందు రాయడం అందుకే.

7. Populating Next Right Pointers in Each Node II (LeetCode #117) — Medium

సమస్య: ఒక binary tree ఇస్తారు — దాని node కి val, left, right తో పాటు ఒక **next** pointer కూడా ఉంది (మొదట అన్నీ **null**). ప్రతి node యొక్క **next** ని **అదే level లో దానికి కుడివైపు ఉన్న తర్వాతి node** కి point చేయాలి. ఒక level లో చివరి node `next = null`. చెట్టు **ఏ ఆకారంలోనైనా** (perfect అవసరం లేదు — అదే #116 తో తేడా).

Constraints: nodes `[0, 6000]`. Follow-up: **constant extra space** వాడాలి (recursion stack తప్ప).

ఉదాహరణ:

Input: <pre> 1 / \ 2 3 / \ \ 4 5 7 </pre>	Output (next pointers): <pre> 1 → null 2 → 3 → null 4 → 5 → 7 → null </pre>
---	--

Node structure:

```

class Node {
    constructor(val = 0, left = null, right = null, next = null) {
        this.val = val; this.left = left; this.right = right; this.next = next;
    }
}

```

ఎలా ఆలోచించాలి:

"అదే level లో పక్క node" అంటే BFS (level order) గుర్తిస్తుంది. నిజంగా, ఒక queue తో level by level వెళ్తూ, ప్రతి level లో వరుస nodes ని next తో కలపడం సులభం — $O(n)$ time, $O(\text{width})$ space. అది సరైన సమాధానమే.

కానీ follow-up **$O(1)$ space** అడుగుతుంది. Trick: **ఒక level ని process చేస్తున్నప్పుడు, ఆ level లో ఇప్పటికే next pointers ఉన్నాయి** (పై level నుండి కట్టేసినవి). ఆ next pointers ని వాడి current level ని ఎడమ నుండి కుడికి **linked-list లా నడవోచ్చు** — queue అవసరం లేదు! నడుస్తూ, ప్రతి node children (left, right) ని **తర్వాతి level యొక్క linked list** గా కడతాం.

తర్వాతి level కట్టడానికి ఒక **dummy head** node వాడతాం (linked list building లో classic trick — head ని ప్రత్యేకంగా handle చేయకుండా). ఒక **tail** pointer తో children ని వరుసగా జోడిస్తాం. Level పూర్తయ్యాక `dummy.next` = తర్వాతి level మొదటి node → అక్కడికి దిగుతాం.

Brute Force / naive (BFS, $O(\text{width})$ space):

Queue తో level order — ప్రతి level ని పూర్తిగా pop చేస్తూ, current ని next లో previous కి కలుపు. సులభం, correct, కానీ $O(n)$ extra space (queue).

- ఎందుకు "సరిపోదు": తప్పు కాదు — కానీ follow-up $O(1)$ space కోరుతుంది. Interview లో ఇది చెప్పి, తర్వాత $O(1)$ కి upgrade చెయ్యి.

Optimal Approach ($O(1)$ space):

Insight: పై level యొక్క **next** chain ని వాడి ఆ level మీద నడు; నడుస్తూ children ని dummy+tail తో తర్వాతి level linked list గా కట్టు.

Plan:

1. leftmost = current level మొదటి node (మొదట root).
2. ప్రతి level కి: dummy (కొత్త node), tail = dummy . cur = leftmost నుండి cur.next వెంట నడు.
3. ప్రతి cur కి: cur.left, cur.right ఉంటే tail.next లో జోడించి tail ముందుకు జరుపు.
4. Level ముగిసాక leftmost = dummy.next (తర్వాతి level మొదటి node). null అయితే ఆగు.

Solution (JavaScript):

```
function connect(root) {  
  let leftmost = root; // ప్రస్తుత level మొదటి node  
  
  while (leftmost !== null) {  
    // తర్వాతి level ని linked list గా కట్టడానికి dummy head + tail  
    const dummy = new Node(0);  
    let tail = dummy;  
  
    // ప్రస్తుత level మీద, ఇప్పటికే ఉన్న next chain వెంట నడు  
    let cur = leftmost;  
    while (cur !== null) {  
      if (cur.left !== null) { tail.next = cur.left; tail = tail.next; }  
      if (cur.right !== null) { tail.next = cur.right; tail = tail.next; }  
      cur = cur.next; // అదే level లో వచ్చే node కి (పై level కట్టిన chain)  
    }  
  
    leftmost = dummy.next; // తర్వాతి level మొదటి node కి దిగు  
  }  
  
  return root;  
}
```

Dry Run: tree [1,2,3,4,5,null,7]

```
Level 1: leftmost=1. dummy→(children of 1: 2,3). dummy.next=2, 2.next=3.  
        leftmost = 2  
Level 2: cur=2 → children 4,5 → dummy.next=4, 4.next=5  
        cur=2.next=3 → child 7 → 5.next=7  
        leftmost = 4  
Level 3: cur=4(leaf), cur=5(leaf), cur=7(leaf) → children లేవు → dummy.next=null
```

leftmost = null → అగు

ఫలితం: 2→3→null, 4→5→7→null ✓

Complexity:

- **Time:** $O(n)$ — ప్రతి node ఒకసారి visit.
- **Space:** $O(1)$ — dummy, tail, cur, leftmost — కొన్ని pointers మాత్రమే. Queue లేదు, recursion లేదు.

గుర్తుంచుకోవాల్సినది:

"పై level యొక్క next chain ని వాడి తర్వాతి level కట్టడం" — tree ని levels గా చూస్తూ $O(1)$ space. **Dummy head + tail** అనే linked-list building trick ఇక్కడ కీలకం (children ని వరుసగా కట్టడానికి). #116 (perfect tree) కి ఇదే code పని చేస్తుంది. BFS solution తెలిసి ఉండి, follow-up కి ఈ $O(1)$ upgrade చెప్పగలిగితే SSE score.

సాధారణ తప్పులు:

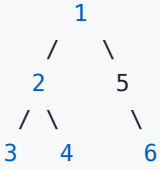
- **Perfect tree అనుకుని node.left.next = node.right రాయడం:** #116 కి పని చేస్తుంది, కానీ #117 (any shape) కి null children ఉంటే విరిగిపోతుంది. Dummy+tail approach shape-agnostic.
- **Dummy ని ప్రతి level కి reset చేయకపోవడం:** ప్రతి level కి కొత్త dummy / tail కావాలి. లేకపోతే levels కలిసిపోతాయి.
- **BFS ని $O(1)$ అనడం:** queue $O(\text{width})$ space — అది $O(1)$ కాదు. Follow-up కి తేడా తెలుసుకో.

8. Flatten Binary Tree to Linked List (LeetCode #114) — Medium

సమస్య: ఒక binary tree root ని **in-place** గా ఒక "linked list" గా చదును (flatten) చెయ్యాలి. నియమాలు: (1) linked list ప్రతి node **right** pointer వాడాలి (next లా), **left** ఎప్పుడూ **null**. (2) nodes క్రమం చెట్టు యొక్క **preorder** traversal క్రమంలో ఉండాలి.

Constraints: nodes $[0, 2000]$. Follow-up: **$O(1)$ extra space** లో చేయగలవా?

ఉదాహరణ:

Input: 

Output: 1 → 2 → 3 → 4 → 5 → 6
(అన్నీ right pointers; left అన్నీ null)

 ... (preorder: 1,2,3,4,5,6)

ఎలా ఆలోచించాలి:

Preorder = root, left, right. కాబట్టి flatten అయ్యాక root తర్వాత మొత్తం left subtree (preorder లో), తర్వాత right subtree రావాలి.

సులభ ఆలోచన (recursion): ప్రతి node దగ్గర – "నా left subtree ని flatten చెయ్యి, నా right subtree ని flatten చెయ్యి. ఇప్పుడు నా right స్థానంలో flattened-left ని పెట్టు, ఆ list చివర్లో flattened-right ని జోడించు, left ని null చెయ్యి." ఇది పని చేస్తుంది కానీ list చివర కనుక్కోవడానికి కొంచెం శ్రమ. దీన్ని ఇంకా చక్కగా **reverse-preorder** (కుడి → ఎడమ → node) తో ఒక prev variable వాడి చేయవచ్చు – కానీ recursion stack $O(h)$.

అందమైన $O(1)$ trick (Morris-style): ప్రతి node cur దగ్గర – cur కి left subtree ఉంటే, ఆ left subtree యొక్క అట్టడుగు కుడి node (rightmost = preorder లో left subtree చివరి node) ని కనిపెట్టు. ఆ node తర్వాత cur.right (అసలు right subtree) రావాలి కదా. కాబట్టి: rightmost.right = cur.right; తర్వాత cur.right = cur.left; cur.left = null. ఇప్పుడు cur కి కుడివైపు దిగు, పునరావృతం. Stack అవసరం లేదు!

Brute Force / naive:

Preorder traversal చేసి nodes ని ఒక array లో collect చేసి, తర్వాత array వెంట node.right = next, node.left = null set చేయడం. Correct, సులభం, కానీ **$O(n)$ extra space** (array). Follow-up $O(1)$ ని miss అవుతుంది.

Optimal Approach ($O(1)$ space, iterative):

Insight: ప్రతి node కి, left subtree ని current node, దాని original right మధ్య "చొప్పించు". Left subtree యొక్క rightmost node = ఆ చొప్పించు కి కుట్టు చీటు.

Plan:

1. cur = root . cur null అయ్యేదాకా:
2. cur.left ఉంటే: prev = cur.left నుండి rightmost దాకా వెళ్ళు (while prev.right: prev = prev.right).
3. prev.right = cur.right (అసలు కుడిని left subtree తోకకి తగిలించు); cur.right = cur.left ; cur.left = null .
4. cur = cur.right (ముందుకు).

Solution (JavaScript):

```
function flatten(root) {
  let cur = root;
  while (cur !== null) {
    if (cur.left !== null) {
      // left subtree లో preorder-చివరి node = దాని rightmost
      let prev = cur.left;
      while (prev.right !== null) prev = prev.right;

      // అసలు కుడి subtree ని ఆ చివరికి తగిలించు
      prev.right = cur.right;
      // left subtree ని కుడికి తరలించు, left ని ఖాళీ చెయ్యి
      cur.right = cur.left;
      cur.left = null;
    }
    cur = cur.right; // ముందుకు (కొత్తగా అమర్చిన chain వెంట)
  }
  // in-place – return అవసరం లేదు (LeetCode void), కావాలంటే return root;
}
```

Dry Run: root = [1,2,5,3,4,null,6]

```
cur=1: left=2 ఉంది. left subtree rightmost = 4.
      4.right = 1.right(=5); 1.right = 2; 1.left = null.
      tree ఇప్పుడు: 1→2→(3,4→5→6...) ... cur = 2
cur=2: left=3. rightmost(3)=3. 3.right = 2.right(=4); 2.right=3; 2.left=null.
      cur = 3
cur=3: left లేదు → cur = 3.right = 4
cur=4: left లేదు → cur = 4.right = 5
cur=5: left లేదు → cur = 5.right = 6
cur=6: left లేదు → cur = null → ఆగు

ఫలితం: 1→2→3→4→5→6 (అన్నీ right, left=null) ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి process; rightmost వెతకడం amortized (ప్రతి edge ఎక్కువలో ఎక్కువ ఒకసారి traverse) → మొత్తం $O(n)$.
- **Space:** $O(1)$ – కేవలం cur, prev pointers. (Naive/recursion $O(n)/O(h)$).

గుర్తుంచుకోవాల్సినది:

"Left subtree ని node, దాని right మధ్య చొప్పించడం; కుట్టు చీటు = left subtree rightmost" – ఇదే Morris-flatten insight. ఈ "predecessor కనిపెట్టి rewire" idea **Morris Traversal** (#94 $O(1)$ inorder) లోనూ వస్తుంది. Naive (preorder array) చెప్పి, $O(1)$ కి upgrade చెప్పడం interview లో బలం.

సాధారణ తప్పులు:

- **cur.left = null మర్చిపోవడం:** left ని null చేయకపోతే tree cyclic/invalid అవుతుంది.
- **rightmost కాకుండా cur.left.right ని వాడటం:** అసలు right ని left subtree లోతైన కుడి చివరికి తగిలించాలి – కేవలం cur.left.right కి కాదు.
- **cur = cur.left అని జరపడం:** rewire తర్వాత left null అయింది; ముందుకు వెళ్ళాల్సింది cur.right వెంట (కొత్త chain).
- **Null root:** while మొదట్లోనే ఆగుతుంది → safe.

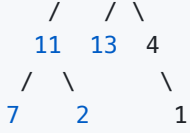
9. Path Sum (LeetCode #112) – Easy

సమస్య: ఒక binary tree root మరియు ఒక సంఖ్య targetSum ఇస్తారు. root నుండి ఏదైనా leaf వరకు ఒక path ఉందా – ఆ path లోని అన్ని node values కలిపితే సరిగ్గా targetSum వచ్చేలా? ఉంటే true, లేకపోతే false.

Constraints: nodes [0, 5000]; $-1000 \leq \text{Node.val} \leq 1000$; $-1000 \leq \text{targetSum} \leq 1000$ (values, target రెండూ negative అవ్వచ్చు – గుర్తుంచుకో).

ఉదాహరణ:

```
  5
 / \
4   8
```



targetSum = 22 → true (దారి 5 → 4 → 11 → 2 = 22)

ఎలా ఆలోచించాలి:

"root నుండి leaf దాకా sum = target" – ఇది top-down ఆలోచన కి సరిపోతుంది. Root వద్ద నుండి కిందకి దిగుతూ, నా node value ని target నుండి తీసివేస్తూ పోతే? అప్పుడు మిగిలిన "కావాల్సిన sum" (remaining) ని children కి పంపుతా.

ఒక node దగ్గర ప్రశ్న: "నా విలువ తీసేశాక మిగిలిన remaining ని, నా children లో ఏదో ఒక subtree పూరించగలదా?" Leaf కి చేరినప్పుడు – ఇక కింద ఎవరూ లేరు – ఆ leaf value సరిగ్గా remaining కి సమానమైతే success. కాబట్టి **leaf దగ్గర**

check: `remaining == leaf.val` ? (సమానం. అంతకుమించి కాదు.)

రెండు కీలక base cases: (1) node null → false (ఇది path కాదు – leaf కాదు, ఖాళీ). (2) leaf (రెండు children null) → targetSum === node.val చూడు. Leaf కాకపోతే remaining పంపి recurse; ఏదో ఒక child true ఇస్తే చాలు (OR).

Optimal Approach:

Insight: target నుండి node.val తీసి children కి remaining పంపు; leaf దగ్గర remaining exact match చూడు; children ల మధ్య OR.

Solution (JavaScript):

```

function hasPathSum(root, targetSum) {
  if (root === null) return false; // ఖాళీ → path లేదు

  // leaf: రెండు children null → ఇక్కడ దారి ముగుస్తుంది
  if (root.left === null && root.right === null) {
    return root.val === targetSum; // మిగిలిన target సరిగ్గా ఈ leaf value అ?
  }

  // leaf కాదు → నా value తీసేసి, మిగిలిన target ని children కి పంపు
  const remaining = targetSum - root.val;
  return hasPathSum(root.left, remaining) || hasPathSum(root.right, remaining);
}

```

Dry Run: target = 22, root = పైన చెట్టు

```

hasPathSum(5, 22): leaf కాదు → remaining = 22-5 = 17
├─ hasPathSum(4, 17): remaining = 17-4 = 13
│   └─ hasPathSum(11, 13): remaining = 13-11 = 2
│       ├── hasPathSum(7, 2): leaf, 7 === 2? false
│       └─ hasPathSum(2, 2): leaf, 2 === 2? TRUE ✓
│           → true
│       → true
└─ → true (|| short-circuit) ✓

```

Complexity:

- **Time:** $O(n)$ – worst case అన్ని nodes visit.
- **Space:** $O(h)$ – recursion stack.

గుర్తుంచుకోవాల్సినది:

"Target నుండి తీసివేస్తూ కిందకి; leaf దగ్గర exact match" – top-down accumulation pattern. **Leaf నిర్వచనం** (రెండు children null) సరిగ్గా వాడటం కీలకం. బంధువులు: **Path Sum II** (#113 – అన్ని paths list చెయ్యి, backtracking), **Path Sum III** (#437 – ఏ node నుండైనా, prefix-sum), **Sum Root to Leaf** (#129 – తర్వాతి problem).

సాధారణ తప్పులు:

- **null ని leaf అనుకోవడం (అతిపెద్ద bug):** ఒక node కి ఒకే child ఉంటే, null child వైపు వెళ్లి "leaf, target-తో పోల్చు" చేస్తే తప్పు. Leaf = **రెండూ null**. null node ప్రత్యేకంగా **return false**. ఉదా: root=[1,2], target=1 → root leaf కాదు (left child 2 ఉంది), null-right వైపు false, left వైపు remaining 0 vs 2 → సరిగ్గా false.
- **target==0 తో అగడం:** negative values ఉండొచ్చు కాబట్టి "remaining 0 అయితే ఆగు" అనే shortcut తప్పు. ఎప్పుడూ **leaf దగ్గరే** అంతిమ నిర్ణయం.
- **Empty tree:** root=null, target=0 → **false** (root-to-leaf path అస్సలు లేదు). పైన null check సరిగ్గా handle.

10. Sum Root to Leaf Numbers (LeetCode #129) – Medium

సమస్య: ప్రతి node లో ఒక అంకె (0-9) ఉన్న binary tree ఇస్తారు. ప్రతి **root-to-leaf path** ఒక సంఖ్యని represent చేస్తుంది (అంకెలు వరుసగా చదివితే). ఉదా: path 1→2→3 = సంఖ్య 123 . అన్ని **root-to-leaf సంఖ్యల మొత్తం** return చెయ్యాలి.

Constraints: nodes [1, 1000] ; 0 ≤ Node.val ≤ 9 ; answer 32-bit range లో ఉంటుంది.

ఉదాహరణ:

```

      1
     / \
    2   3
  
```

paths: 1→2 = 12, 1→3 = 13
sum = 12 + 13 = 25 → Output: 25

ఎలా ఆలోచించాలి:

Path అంటే అంకెల వరుస → సంఖ్య. 1→2→3 ని 123 గా ఎలా కడతాం? అంకె అంకెగా: మొదట 1, తర్వాత $1 \times 10 + 2 = 12$, తర్వాత $12 \times 10 + 3 = 123$. అంటే కిందకి దిగుతున్నప్పుడు, **ఇప్పటిదాకా కట్టిన సంఖ్య cur ని $cur \times 10 + node.val$ గా update చేస్తూ పోవాలి.** ఇది #112 (Path Sum) లాంటిదే – target తీసివేయడం బదులు, ఇక్కడ సంఖ్య కడుతున్నాం (top-down).

Leaf కి చేరినప్పుడు cur = ఆ path పూర్తి సంఖ్య → దాన్ని return. Leaf కాకపోతే left, right subtrees నుండి వచ్చిన totals ని **కలుపు** (ప్రతి leaf ఒక సంఖ్యని contribute చేస్తుంది; మనకి అన్నిటి sum కావాలి). Base case: null → 0 (contribute చేయదు).

Optimal Approach:

Insight: DFS లో `cur` (ఇప్పటిదాకా అంకెలు) ని accumulator గా carry చెయ్యి. Leaf $\rightarrow$ cur return. Internal $\rightarrow$ left+right subtree sums.

Solution (JavaScript):

```
function sumNumbers(root) {  
  // cur = root నుండి ఈ node parent దాకా కట్టిన సంఖ్య  
  function dfs(node, cur) {  
    if (node === null) return 0; // ఖాళీ  $\rightarrow$  contribute చేయదు  
  
    cur = cur * 10 + node.val; // ఈ node అంకెని చేర్చు  
  
    // leaf  $\rightarrow$  ఈ path సంఖ్య పూర్తయింది  
    if (node.left === null && node.right === null) return cur;  
  
    // internal  $\rightarrow$  రెండు subtrees నుండి వచ్చే అన్ని path సంఖ్యల sum  
    return dfs(node.left, cur) + dfs(node.right, cur);  
  }  
  return dfs(root, 0);  
}
```

Dry Run: root = [4,9,0,5,1]



```
dfs(4, 0): cur = 0*10+4 = 4. internal  $\rightarrow$  left + right  
├─ dfs(9, 4): cur = 4*10+9 = 49. internal  $\rightarrow$  left + right  
│   ├─ dfs(5, 49): cur = 495. leaf  $\rightarrow$  495  
│   └─ dfs(1, 49): cur = 491. leaf  $\rightarrow$  491  
│   └─  $\rightarrow 495 + 491 = 986$   
└─ dfs(0, 4): cur = 40. leaf  $\rightarrow$  40  
└─  $\rightarrow 986 + 40 = 1026$  ✓ (495 + 491 + 40)
```

Complexity:

- **Time:** $O(n)$ — ప్రతి node ఒకసారి.
- **Space:** $O(h)$ — recursion stack.

గుర్తుంచుకోవాల్సినది:

"Path-గా carry చేసే accumulator" (top-down state) pattern — #112, #129 ఒకే కుటుంబం. తేడా: #112 target తగ్గిస్తుంది, #129 సంఖ్య కడుతుంది ($cur*10+val$). Leaf దగ్గర ఆ path viresult ఇచ్చి, internal దగ్గర children results ని combine చేయడం. ఇలాంటి "cur ని పరామితిగా పంపే" DFS చాలా tree problems లో వస్తుంది (path collection, string building).

సాధారణ తప్పులు:

- **null దగ్గర cur return చేయడం:** null ని leaf అనుకుని cur return చేస్తే, ఒంటి-child nodes కి path రెట్టింపు లెక్కవుతుంది. $null \rightarrow 0$.
- **Global variable తో గందరగోళం:** ఒక outer total వాడి leaf దగ్గర add చేయడం కూడా పని చేస్తుంది, కానీ pure-return style (ప్రైనలాగా) cleaner, side-effect-free.
- **cur ని string గా కట్టి చివర్లో parseInt:** పని చేస్తుంది కానీ నెమ్మది + extra memory. Arithmetic ($*10+val$) సాగసు.

11. Binary Tree Maximum Path Sum (LeetCode #124) — Hard

సమస్య: ఒక binary tree root ఇస్తారు. ఏదైనా **non-empty path** యొక్క node values మొత్తం — వాటిలో **గరిష్టమైనది** return చెయ్యాలి. ఇక్కడ **path** అంటే: nodes వరుస, ప్రతి వరుస జతకి మధ్య edge ఉంది, ప్రతి node ఎక్కువలో ఎక్కువ ఒకసారి. Path **root గుండా వెళ్ళాల్సిన అవసరం లేదు**, leaf దగ్గర మొదలవ్వాల్సిన/ముగియాల్సిన అవసరం లేదు — చెట్టులో ఏ node నుండి ఏ node వరకైనా (ప్రెకీ-కిందకి తిరిగే ఒక్క "V" ఆకారం).

Constraints: nodes $[1, 3 \times 10^4]$; $-1000 \leq \text{Node.val} \leq 1000$ (negatives కీలకం).

ఉదాహరణ:



Input1: $[-10, 9, 20, null, null, 15, 7]$ → 42 (path 15 → 20 → 7)
 Input2: $[1, 2, 3]$ → 6 (path 2 → 1 → 3)
 Input3: $[-3]$ → -3 (ఒక్క node — non-empty కావాలి)

ఎలా ఆలోచించాలి:

ఇది Hard ఎందుకంటే రెండు వేర్వేరు ఆలోచనలు కలవాలి. ఒక node దగ్గర ఆలోచించు. ఈ node ద్వారా వెళ్ళే **best path** ఒక "V" (లేదా "^") ఆకారంలో ఉండొచ్చు: **ఎడమ నుండి పైకి → ఈ node → కుడికి కిందకి**. ఆ path sum = node.val + (ఎడమ కొమ్మ నుండి వచ్చే **best** downward gain) + (కుడి కొమ్మ నుండి వచ్చే **best** downward gain). ఇది ఈ node "peak" (శిఖరం) గా ఉన్న best path. దీన్ని global answer తో పోల్చి update చేస్తాం.

కానీ ఒక కీలక తేడా: ఈ node తన **parent కి పైకి report చేసేటప్పుడు**, V ఆకారం report చేయలేదు — ఎందుకంటే parent కి కలిస్తే path ఒకే సరళ రేఖలా (ఒక కొమ్మ మాత్రమే) కొనసాగాలి, రెండు కొమ్మలూ కాదు. కాబట్టి node తన parent కి **"నా నుండి కిందకి ఒక్క కొమ్మలో వచ్చే max gain"** = node.val + max(ఎడమ gain, కుడి gain) ఇస్తుంది.

రెండు వేర్వేరు విలువలు:

1. **global best (answer):** node.val + leftGain + rightGain (రెండు కొమ్మలూ — ఈ node peak).
2. **return value (parent కి):** node.val + max(leftGain, rightGain) (ఒక కొమ్మ మాత్రమే).

Negative gains ని **0 కి clamp** చెయ్యి — ఒక కొమ్మ negative contribute చేస్తే, ఆ కొమ్మని వదిలేయడమే మంచిది (Math.max(gain, 0)). Base case: null → 0 gain.

Brute Force / naive:

ప్రతి node ని "peak" గా తీసుకుని, దాని నుండి ఎడమ/కుడి కి longest downward path లెక్కించడం – ప్రతి node కి $O(n)$ పని $\rightarrow O(n^2)$. $n = 30000$ కి నెమ్మది. Downward gains ని ఒక postorder pass లో recompute చేయకుండా వాడితే $O(n)$.

Optimal Approach (postorder, $O(n)$):

Insight: ఒక postorder DFS `gain(node)` – parent కి "single-branch downward max gain" return చేస్తుంది; పక్కనే ఒక outer `best` variable లో "ఈ node peak గా ఉన్న two-branch path" ని update చేస్తుంది.

Plan:

1. `best = -Infinity` (అన్నీ negative అయినా సరైన answer కోసం).
2. `gain(node) : null  $\rightarrow$  0`. `left = max(gain(node.left), 0)`, `right = max(gain(node.right), 0)`.
3. `best = max(best, node.val + left + right)` (two-branch, peak here).
4. `return node.val + max(left, right)` (single branch upward).

Solution (JavaScript):

```
function maxPathSum(root) {
  let best = -Infinity; // answer (path కనీసం ఒక node కాబట్టి -Infinity నుండి)

  // node నుండి కిందకి ఒక్క కొమ్మలో వచ్చే max gain ని return చేస్తుంది
  function gain(node) {
    if (node === null) return 0;

    // negative కొమ్మని వదిలేయడం మంచిది  $\rightarrow$  0 కి clamp
    const left = Math.max(gain(node.left), 0);
    const right = Math.max(gain(node.right), 0);

    // ఈ node "శిఖరం"గా ఉన్న path (రెండు కొమ్మలూ) – answer candidate
    best = Math.max(best, node.val + left + right);

    // parent కి: ఒక్క కొమ్మ మాత్రమే కొనసాగగలదు
    return node.val + Math.max(left, right);
  }

  gain(root);
  return best;
}
```

Dry Run: `[-10, 9, 20, null, null, 15, 7]`

<code>gain(9): left=0, right=0 $\rightarrow$ best=max($-\infty$, 9)=9;</code>	<code>return 9</code>
<code>gain(15): $\rightarrow$ best=max(9, 15)=15;</code>	<code>return 15</code>
<code>gain(7): $\rightarrow$ best=max(15, 7)=15;</code>	<code>return 7</code>
<code>gain(20): left=max(15, 0)=15, right=max(7, 0)=7</code> <code>best = max(15, 20+15+7=42) = 42;</code>	<code>return 20+max(15, 7)=35</code>
<code>gain(-10): left=max(9, 0)=9, right=max(35, 0)=35</code> <code>best = max(42, -10+9+35=34) = 42;</code>	<code>return -10+35=25</code>
<code>answer = best = 42</code>  (path 15 $\rightarrow$ 20 $\rightarrow$ 7)	

Input3 [-3] : gain(-3): left=0,right=0 → best=max(-∞, -3) = -3 → answer -3 ✓ (0 కి clamp చేయకపోవడం వల్ల single negative node కూడా సరిగ్గా వస్తుంది).

Complexity:

- **Time:** $O(n)$ — ఒకే postorder pass.
- **Space:** $O(h)$ — recursion stack.

గుర్తుంచుకోవాల్సినది:

"రెండు వేర్వేరు విలువలు: node peak గా answer update, పానీ parent కి single-branch return" — ఇది చాలా Hard tree problems కి master pattern. **Diameter of Binary Tree** (#543 — అదే నిర్మాణం, val బదులు edge counts), **Longest Univalued Path, Binary Tree Cameras** అన్నీ ఈ "postorder లో local answer update + upward report" idea. Negative clamp ($\max(\text{gain}, 0)$) trick గుర్తుంచుకో.

సాధారణ తప్పులు:

- **best init 0:** అన్నీ negative అయితే ([-3]) 0 తప్పు answer. -Infinity వాడు. (Node values 0 కి clamp చేయకూడదు — కొమ్మ gains మాత్రమే.)
- **Return లో రెండు కొమ్మలూ కలపడం:** `return node.val + left + right` రాస్తే tree cyclic path అవుతుంది (parent కి కలిస్తే V రెండేసారి). Return లో $\max(\text{left}, \text{right})$ ఒక్కటే.
- **Negative gain clamp మర్చిపోవడం:** clamp లేకపోతే negative subtree ని బలవంతంగా చేర్చి answer తగ్గుతుంది.
- **Answer ని return value నుండి తీసుకోవడం:** `gain(root)` return చేసేది single-branch — అది answer కాదు. Answer outer best .

12. Binary Search Tree Iterator (LeetCode #173) — Medium

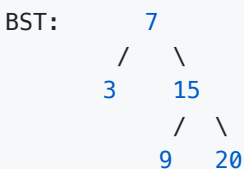
సమస్య: ఒక BST మీద inorder traversal ని అనుకరించే BSTIterator class రాయాలి:

- `constructor(root)` — root తో initialize.
- `next()` — inorder క్రమంలో తర్వాతి అతిచిన్న element return చేసి ముందుకు జరుగు.
- `hasNext()` — ఇంకా elements ఉన్నాయా (`true` / `false`).

Follow-up: `next()` , `hasNext()` **average $O(1)$** time; **$O(h)$** memory (h = tree height) లో చెయ్యగలవా? (మొత్తం inorder ని ముందే array లో పెట్టడం $O(n)$ memory — దాన్ని తప్పించాలి.)

Constraints: nodes [1, 10^5] ; `next()` valid గా ఉన్నప్పుడే (`hasNext true`) పిలుస్తారు; మొత్తం `next` calls $\leq$ nodes.

ఉదాహరణ:



```
next()→3, next()→7, hasNext()→true, next()→9, next()→15,  
hasNext()→true, next()→20, hasNext()→false  
(inorder = 3,7,9,15,20 – BST లో inorder ఎప్పుడూ sorted ascending!)
```

ఎలా ఆలోచించాలి:

సులభ ఆలోచన: constructor లోనే మొత్తం inorder traversal చేసి ఒక array లో పెట్టి, ఒక index తో next() ఇవ్వడం. next/hasNext నిజంగా $O(1)$, కానీ **$O(n)$ memory** (మొత్తం array) – follow-up $O(h)$ ని miss.

Follow-up ని ఎలా? Inorder traversal recursion ని గుర్తు తెచ్చుకో: "ముందు ఎడమకి అట్టడుగు దాకా వెళ్ళు (leftmost = అతిచిన్నది), ఆ node ఇవ్వు, తర్వాత దాని right subtree లోకి." ఈ recursion ని **stack తో iterative** గా, పైగా **pause/resume చేయగలిగేలా** రాస్తే – అదే iterator!

Stack లో "ఇంకా process చేయాల్సిన ancestors" ని ఉంచుతాం. Init లో root నుండి **leftmost దాకా అన్నీ nodes push** (అవి inorder లో ముందు వచ్చేవి, అట్టడుగు = అతిచిన్నది top మీద). **next()** : stack top ని pop (అదే తర్వాతి అతిచిన్నది). ఆ node కి **right child ఉంటే**, ఆ right నుండి మళ్ళీ leftmost దాకా push (ఇప్పుడు ఆ subtree లో అతిచిన్నది తర్వాత రావాలి). **hasNext()** : stack ఖాళీ కాదా.

ఎందుకు $O(h)$ memory? Stack లో ఎప్పుడూ ఒక root-to-node path మాత్రమే (ancestors), అది ఎక్కువలో ఎక్కువ height h .

Optimal Approach:

Insight: Controlled iterative inorder. Stack = "pending leftmost spine". **_pushLeft(node)** helper node నుండి left వెంట అన్నీ push. next = pop + (right ఉంటే) **_pushLeft(right)**.

Solution (JavaScript):

```
class BSTIterator {  
  constructor(root) {  
    this.stack = [];  
    this._pushLeft(root); // root నుండి leftmost దాకా push (అతిచిన్నది top)  
  }  
  
  // node నుండి ఎడమ వెంట అన్నీ nodes ని stack లో పెట్టు  
  _pushLeft(node) {  
    while (node !== null) {  
      this.stack.push(node);  
      node = node.left;  
    }  
  }  
  
  next() {  
    const node = this.stack.pop(); // తర్వాతి అతిచిన్నది  
    // ఈ node right subtree లో అతిచిన్నది ఇప్పుడు రావాలి → వాటి leftmost spine push  
    if (node.right !== null) this._pushLeft(node.right);  
    return node.val;  
  }  
  
  hasNext() {  
    return this.stack.length > 0;  
  }  
}
```

```
}
}
```

Dry Run: ఫైన BST [7, 3, 15, null, null, 9, 20]

```
init: _pushLeft(7): push 7, push 3 (3.left=null అగు) → stack=[7,3]
next(): pop 3. 3.right=null. return 3          stack=[7]
next(): pop 7. 7.right=15 → _pushLeft(15): push 15, push 9  stack=[15,9]; return 7
hasNext(): true
next(): pop 9. 9.right=null. return 9          stack=[15]
next(): pop 15. 15.right=20 → push 20          stack=[20]; return 15
next(): pop 20. 20.right=null. return 20       stack=[]
hasNext(): false ✓ → వరుస: 3,7,9,15,20
```

Complexity:

- **Time:** `next()` — **amortized $O(1)$** . ఒక్కో `next` లో while loop ఎక్కువ పని చేయొచ్చు, కానీ ప్రతి node జీవితకాలంలో ఒకసారి push, ఒకసారి pop → n calls మీద మొత్తం $O(n)$ → సగటున $O(1)$. `hasNext()` $O(1)$.
- **Space:** $O(h)$ — stack లో ఒక path ancestors మాత్రమే.

గుర్తుంచుకోవాల్సినది:

"Recursion ని stack తో pause/resume చేయగల iterator గా మార్చడం" — ఇది strong pattern. "Leftmost spine ని push, pop చేసి right కి దిగు" అనేదే iterative inorder core (BST kth smallest #230 కూడా ఇదే — తర్వాత చూద్దాం). Amortized $O(1)$ reasoning (ప్రతి node ఒకే push/pop) interview లో స్పష్టంగా చెప్పు.

సాధారణ తప్పులు:

- **మొత్తం array ముందే build చేయడం:** correct కానీ $O(n)$ memory — follow-up miss. Stack approach $O(h)$.
- **pop తర్వాత right ని push మర్చిపోవడం:** అప్పుడు right subtree elements ఎప్పటికీ రావు → iteration అసంపూర్ణం.
- **_pushLeft లో right కూడా push చేయడం:** ఎడమ spine మాత్రమే push. Right ని పైకి వచ్చినప్పుడు (దాని node pop అయ్యాక) handle చేస్తాం.
- **hasNext ని పిలవకుండా next అతిగా పిలవడం:** ఖాళీ stack మీద pop → undefined.val crash. Constraints ప్రకారం hasNext true అయితేనే next.

13. Count Complete Tree Nodes (LeetCode #222) — Easy

సమస్య: ఒక **complete binary tree** root ఇస్తారు. మొత్తం nodes సంఖ్య return చెయ్యాలి. **Complete tree** అంటే: చివరి level తప్ప అన్ని levels **పూర్తిగా నిండి** ఉంటాయి, చివరి level nodes అన్నీ **ఎడమవైపు నుండి** వరుసగా ఉంటాయి (మధ్యలో gaps ఉండవు).

Constraints: nodes [0, $5 \cdot 10^4$] . Follow-up: $O(n)$ కన్నా వేగంగా — completeness ని exploit చేసి.

ఉదాహరణ:



Input: [1,2,3,4,5,6] → Output: 6

ఎలా ఆలోచించాలి:

Naive: ప్రతి node ని recursion తో లెక్కపెట్టు - $1 + \text{count}(\text{left}) + \text{count}(\text{right})$, $O(n)$. పని చేస్తుంది. కానీ "complete tree" అనే మాట ఒక కానుక - దాన్ని వాడకపోతే వృథా.

గమనించు: ఒక tree **perfect** అయితే (అన్ని levels పూర్తిగా నిండి, height h), దాని nodes = $2^h - 1$ (సూత్రం, ఒక్క $O(h)$ లెక్కింపు). ఉదా: height 3 perfect → 7 nodes.

Perfect ఆ కాదా ఎలా తెలుసు? **ఎడమ అంచు** వెంట లోతు (leftHeight) మరియు **కుడి అంచు** వెంట లోతు (rightHeight) కొలుచు. అవి **సమానం అయితే** - complete tree లో అది perfect subtree (అన్నీ నిండాయి) → $2^h - 1$ నేరుగా. అవి **తేడా అయితే** - చివరి level సగం నిండింది → సాధారణ recursion: $1 + \text{count}(\text{left}) + \text{count}(\text{right})$. కానీ ఇక్కడ magic: complete tree లో left, right subtrees లో **ఒకటి తప్పకుండా perfect** - కాబట్టి recursion ఒక వైపు వెంటనే సూత్రంతో ముగుస్తుంది, మరో వైపు లోతుకి దిగుతుంది. ఫలితం: $O(\log n)$ recursion depth × ఒక్క స్థాయిలో $O(\log n)$ height లెక్కింపు = $O(\log^2 n)$.

Brute Force / naive:

```
function countNodesNaive(root) {
  if (root === null) return 0;
  return 1 + countNodesNaive(root.left) + countNodesNaive(root.right);
}
```

$O(n)$ time - completeness ని వాడలేదు. $n = 50000$ కి పరవాలేదు, కానీ $O(\log^2 n)$ చేయగలం.

Optimal Approach ($O(\log^2 n)$):

Insight: leftHeight == rightHeight అయితే perfect → $2^h - 1$. లేకపోతే recurse (ఒక subtree వెంటనే సూత్రంతో ముగుస్తుంది).

Solution (JavaScript):

```
function countNodes(root) {
  if (root === null) return 0;

  const lh = leftHeight(root); // ఎడమ అంచు వెంట లోతు
  const rh = rightHeight(root); // కుడి అంచు వెంట లోతు

  if (lh === rh) {
    // perfect subtree - nodes = 2^lh - 1 (bit shift తో వేగంగా)
    return (1 << lh) - 1;
  }
  // లేకపోతే సాధారణ recursion (ఒక వైపు వెంటనే perfect అవుతుంది)
```

```

    return 1 + countNodes(root.left) + countNodes(root.right);
}

function leftHeight(node) {
    let h = 0;
    while (node !== null) { h++; node = node.left; } // ఎడమ వెంట
    return h;
}
function rightHeight(node) {
    let h = 0;
    while (node !== null) { h++; node = node.right; } // కుడి వెంట
    return h;
}

```

Dry Run: root = [1,2,3,4,5,6]

```

countNodes(1): leftHeight = 1→2→4 = 3; rightHeight = 1→3→(3.right=null) = 2
3 !== 2 → recurse: 1 + countNodes(2) + countNodes(3)
countNodes(2): lh(2→4)=2, rh(2→5)=2 → సమానం → 2^2 - 1 = 3 (perfect: 2,4,5)
countNodes(3): lh(3→6)=2, rh(3→null)=1 → తేడా → 1 + countNodes(6) + countNodes(null)
    countNodes(6): lh=1, rh=1 → 2^1-1 = 1
        → 1 + 1 + 0 = 2 (nodes 3,6)
    → 1 + 3 + 2 = 6 ✓

```

గమనించు: `countNodes(2)` ఒక్క సూత్రంతో ముగిసింది (perfect), పూర్తి recursion అవలేదు.

Complexity:

- **Time: $O(\log^2 n)$** – recursion depth $O(\log n)$; ప్రతి స్థాయిలో leftHeight/rightHeight $O(\log n)$.
- **Space: $O(\log n)$** – recursion stack (complete tree height = $\log n$).

గుర్తుంచుకోవాల్సినది:

"Structure guarantee (completeness) ని exploit చేసి brute $O(n)$ ని $O(\log^2 n)$ కి తగ్గించడం." Perfect subtree = $2^h - 1$ సూత్రం, left/right edge heights పోల్చడం – ఈ trick గుర్తుంచుకో. Interview లో $O(n)$ చెప్పి, "complete అని ఉంది కదా, $O(\log^2 n)$ చేయవచ్చు" అనడం SSE signal. Bit shift $1 \ll h = 2^h$ (h చిన్నది కాబట్టి safe).

సాధారణ తప్పులు:

- **Completeness ని వాడకపోవడం:** $O(n)$ తప్పు కాదు కానీ follow-up ఆశించేది $O(\log^2 n)$.
- **leftHeight ని left వెంట, rightHeight ని right వెంట కాకుండా గందరగోళం:** perfect check కి ఎడమ-అంచు లోతు vs కుడి-అంచు లోతు. తప్పు దిశ → తప్పు perfect నిర్ణయం.
- **2^h bit overflow:** JS లో $1 \ll 31$ sign-flip అవుతుంది, కానీ ఇక్కడ $h \leq \sim 17$ ($n \leq 5 \cdot 10^4$) కాబట్టి safe. చాలా లోతైన trees కి $2 \ll h$ (BigInt కాదు, Number) వాడొచ్చు.
- **Null root:** మొదటి line `return 0` – handle అయింది.

14. Lowest Common Ancestor of a Binary Tree (LeetCode #236) — Medium

సమస్య: ఒక binary tree root మరియు రెండు nodes p , q ఇస్తారు. వాటి **LCA (Lowest Common Ancestor)** return చెయ్యాలి — అంటే p , q ఇద్దరికీ ancestor అయిన nodes లో **అట్టడుగుది (root కి దూరంగా, చెట్టులో లోతైనది)**. ఒక node తనకి తానే ancestor కావచ్చు (p , q లలో ఒకటి మరొకదాని ancestor అయితే, అదే LCA).

Constraints: nodes $[2, 10^5]$; values unique; p , q రెండూ చెట్టులో **తప్పకుండా ఉంటాయి**. (ఇది సాధారణ binary tree — BST కాదు, కాబట్టి value comparison వాడలేం.)

ఉదాహరణ:



LCA(5, 1) = 3 (వేర్వేరు subtrees లో → వాటి కలయిక root)
LCA(5, 4) = 5 (4 అనేది 5 కింద ఉంది → 5 తనకి తానే ancestor)

ఎలా ఆలోచించాలి:

ఒక node దగ్గర నిలబడి ఆలోచించు: "నా subtree లో p , q ఎక్కడ ఉన్నారు?" మూడు అవకాశాలు:

1. p , q ఇద్దరూ నా ఎడమ subtree లో → LCA ఎడమవైపు లోతులో. నేను కాదు.
2. ఇద్దరూ నా కుడి subtree లో → LCA కుడివైపు.
3. ఒకరు ఎడమ, ఒకరు కుడి (లేదా ఒకరు నేనే) → **నేనే LCA!** (ఇక్కడ ఇద్దరూ కలుస్తారు; ఇంతకన్నా కిందకి వెళ్తే ఒకరు మిస్ అవుతారు).

దీన్ని recursion గా ఎలా? ప్రతి node ని అడుగు: "నీ subtree లో p లేదా q ఎవరైనా కనిపించారా? కనిపిస్తే ఎవరో ఒకరిని (లేదా LCA ని) తిరిగి ఇవ్వు." Base case: node null → null (ఎవరూ లేరు). node == p లేదా node == q → **ఆ node నే return** (కనిపించాడు — తనకి తానే ancestor కావచ్చు కాబట్టి కిందకి వెతకనవసరం లేదు).

ఒక node కి ఎడమ recursion **non-null** ఇచ్చి, కుడి recursion కూడా **non-null** ఇస్తే → p ఒకవైపు, q మరోవైపు కనిపించారు → **ఈ node నే LCA**. ఒక్క వైపు non-null అయితే → ఇద్దరూ (లేదా వారికి కనిపించిన LCA) ఆ వైపు ఉన్నారు → ఆ non-null ని పైకి పంపు.

Optimal Approach (single postorder pass):

Insight: postorder recursion; null/found-node ని పైకి propagate; రెండు వైపులా non-null కలిస్తే current node = LCA.

Solution (JavaScript):

```
function lowestCommonAncestor(root, p, q) {  
  // base: ఖాళీ, లేదా p/q లలో ఒకరు దొరికారు → ఈ nodeని పైకి పంపు
```

```

if (root === null || root === p || root === q) return root;

const left = lowestCommonAncestor(root.left, p, q); // ఎడమలో ఏం కనిపించింది?
const right = lowestCommonAncestor(root.right, p, q); // కుడిలో ఏం కనిపించింది?

// రెండు వైపులా కనిపిస్తే → p ఒకవైపు, q మరొకవైపు → నేనే LCA
if (left !== null && right !== null) return root;

// లేకపోతే: non-null వైపు (ఇద్దరూ అక్కడే) పైకి పంపు; రెండూ null అయితే null
return left !== null ? left : right;
}

```

Dry Run: LCA(5, 1) పైన చెట్టులో

```

lca(3): 3≠p,q → left=lca(5..), right=lca(1..)
├ lca(5): 5===p → return 5 (node 5)
└ lca(1): 1===q → return 1 (node 1)
left=5(non-null), right=1(non-null) → రెండూ కనిపించాయి → return 3 ✓

```

LCA(5, 4):

```

lca(3): left=lca(5..), right=lca(1..)
├ lca(5): 5===p → return 5 (కింద 4 ఉన్నా, p కనిపించగానే ఆగుతాం)
└ lca(1): 1 subtree లో 4 లేదు → null
left=5, right=null → non-null (left) పంపు → 3 కి 5 వస్తుంది → చివరికి 5 ✓

```

గమనించు: 5 దగ్గర return చేశాక, 4 ని విడిగా వెతకలేదు – p (5) అనేది q (4) కి ancestor కాబట్టి 5 సరైన answer.

Complexity:

- **Time:** $O(n)$ – ఒకే postorder pass, ప్రతి node ఒకసారి.
- **Space:** $O(h)$ – recursion stack.

గుర్తుంచుకోవాల్సినది:

"రెండు వైపులా కనిపిస్తే current = LCA; ఒక వైపు అయితే ఆ వైపు పైకి పంపు" – ఈ postorder "boundary where two searches meet" pattern. p, q తప్పకుండా ఉంటారు అనే హామీ వల్ల ఈ simple version సరిపోతుంది. (వారు లేకపోవచ్చు అంటే – ఇద్దరూ నిజంగా found అయ్యారా అని రెండు flags track చేయాలి.) **BST అయితే** (#235) value comparison తో ఇంకా సులభం (p, q రెండూ node కన్నా చిన్నవైతే left, పెద్దవైతే right, లేకపోతే current). కానీ ఇది general tree.

సాధారణ తప్పులు:

- **Value (root.val === p.val) తో compare చేయడం:** values unique కాబట్టి ఇక్కడ పని చేస్తుంది, కానీ nodes duplicate అయ్యే problems లో **reference (root === p)** వాడాలి. అలవాటు reference.
- **root === p || root === q దగ్గర కిందకి వెతకడం:** p కనిపించగానే return చేయాలి – ఇద్దరూ ఒకే కొమ్మలో ఒకరు మరొకరి ancestor అయితే, పైవాడే LCA. కిందకి వెతికితే తప్పు.
- **రెండూ ఒకే subtree లో ఉన్నప్పుడు:** left non-null, right null → non-null పంపడం సరైనది; తొందరపడి "ఏదో ఒకటి కనిపించింది కాబట్టి current LCA" అనకూడదు. **రెండూ non-null** అయితేనే current LCA.

Pattern: Tree BFS

వివరణ

BFS (Breadth-First Search) అంటే చెట్టుని **level by level** (అంతస్తు అంతస్తుగా) చూడటం. DFS ఒక కొమ్మని లోతుకి వెళ్తే, BFS ఒక level లోని **అన్ని nodes ని** ముందు చూసి, తర్వాత next level కి దిగుతుంది. Root level 0, దాని children level 1, వాళ్ళ children level 2...

BFS కి ఆయుధం **queue** (FIFO — First In, First Out). ఆలోచన: root ని queue లో పెట్టు. తర్వాత queue నుండి ఒక node తీసి (dequeue), దాని children ని queue లోకి పెట్టు (enqueue). ఇలా చేస్తే nodes సరిగ్గా **level order** లో బయటికి వస్తాయి — ఎందుకంటే level k nodes అన్నీ level k+1 nodes కన్నా ముందు queue లోకి వెళ్తాయి.

కీలక మెలిక — "level by level" గా విడగొట్టడం: చాలా problems కి కేవలం level order కాదు, **ప్రతి level ని విడిగా** కావాలి (level average, right side view, zigzag). Trick: loop మొదట్లో `size = queue.length` గుర్తుంచుకో — **ఆ క్షణం queue లో ఉన్నవి సరిగ్గా current level nodes**. అప్పుడు `size` సార్లు మాత్రమే dequeue చేస్తే ఒక్క level పూర్తవుతుంది; ఈలోపు enqueue అయిన children next level కి మిగులుతాయి. ఈ "snapshot the size" idea BFS level problems అన్నిటికీ ఆత్మ.

JavaScript లో queue కి **plain array** వాడతాం: **push** = enqueue (వెనుక), **shift** = dequeue (ముందు). (గమనిక: `shift()` $O(n)$ — పెద్ద inputs కి index pointer **head** వాడి $O(1)$ చేయవచ్చు; కింద చూద్దాం.)

Real-life Scenario

ఒక **apartment building లో అగ్నిప్రమాదం** జరిగిందని అనుకో. Fire warden ప్రతి floor ని **వరుసగా** ఖాళీ చేయిస్తాడు — ground floor అందరినీ మొదట, తర్వాత 1st floor, తర్వాత 2nd... ఒక floor మొత్తం పూర్తయ్యాకే పైకి. ఒకే floor ని (level ని) పూర్తిగా handle చేసి తర్వాత level కి వెళ్ళడం — అదే BFS. Queue = "ఇంకా ఖాళీ చేయాల్సిన వాళ్ళ వరుస".

ఇంకో analogy: **నీటిలో రాయి వేస్తే ripples** — center (root) నుండి బయటికి వృత్తాలు వృత్తాలుగా (levels) వ్యాపిస్తాయి. దగ్గరివి ముందు, దూరంవి తర్వాత. "ఎంత దూరం / ఎన్ని levels" అనే ప్రశ్నలకి BFS సహజం.

ఎలా గుర్తించాలి (Recognition Signals)

- "level / row / అంతస్తు" అనే మాట — "level order", "each level", "level averages".
- "right side view / left view" — ప్రతి level చివరి/మొదటి node.
- "minimum depth / nearest / closest" — BFS మొదట కనిపించిన leaf = అతిదగ్గర (unweighted shortest path).
- "zigzag / spiral" — levels ని మారుస్తూ చదవడం.
- Level-wise ఏదైనా **aggregate** (sum, max, count per level).

Template Code (JavaScript)

```
// level-by-level BFS — ప్రతి tree BFS problem కి ఇదే అస్థిపంజరం
function bfsLevels(root) {
```

```

const result = [];
if (root === null) return result;

const queue = [root]; // root తో మొదలు
while (queue.length > 0) {
  const size = queue.length; // ★ ఈ క్షణం queue = current level మొత్తం
  const level = [];

  for (let i = 0; i < size; i++) { // సరిగ్గా ఈ level nodes మాత్రమే process
    const node = queue.shift(); // ముందు నుండి తీయి (FIFO)
    level.push(node.val); // ← ఇక్కడ problem-specific పని

    if (node.left) queue.push(node.left); // children ని next level కి
    if (node.right) queue.push(node.right);
  }
  result.push(level); // ఒక level పూర్తయింది
}
return result;
}

// shift() O(n) ను తప్పించే O(1) queue (పెద్ద inputs కి): index pointer వాడు
function bfsFast(root) {
  if (!root) return [];
  const queue = [root];
  let head = 0; // shift బదులు head++ (array copy లేదు)
  const out = [];
  while (head < queue.length) {
    const size = queue.length - head;
    for (let i = 0; i < size; i++) {
      const node = queue[head++];
      out.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
  }
  return out;
}

```

Complexity

- **Time: $O(n)$** – ప్రతి node ఒకసారి enqueue + dequeue. (Plain `shift()` వాడితే $O(n^2)$ worst case – దాన్ని `head` pointer తో $O(n)$ చేయవచ్చు.)
- **Space: $O(w)$** – queue పరిమాణం = tree యొక్క గరిష్ట **width** (ఒక level లో ఎక్కువమంది nodes). Balanced tree కి చివరి level లో $\approx n/2$ nodes $\rightarrow O(n)$. ఇది DFS ($O(h) = O(\log n)$) కన్నా ఎక్కువ memory – trade-off గుర్తుంచుకో: **BFS levels కి సహజం కాన width-space; DFS లోతుకి సహజం కాన height-space.**

15. Binary Tree Right Side View (LeetCode #199) – Medium

సమస్య: ఒక binary tree root ఇస్తారు. చెట్టుని కుడివైపు నుండి చూస్తే కనిపించే nodes ని పైనుండి కిందకి క్రమంలో return చెయ్యాలి. అంటే ప్రతి level లో అట్టడుగు కుడి (rightmost) node.

Constraints: nodes [0, 100] ; -100 <= Node.val <= 100 .

ఉదాహరణ:

```
      1      <- కుడి నుండి చూస్తే 1 కనిపిస్తుంది
     /  \
    2    3    <- 3 కనిపిస్తుంది (2 దాని వెనుక దాక్కుంది)
     \    \
      5    4    <- 4 కనిపిస్తుంది
```

Input: [1,2,3,null,5,null,4] → Output: [1,3,4]

ఎలా ఆలోచించాలి:

"కుడి నుండి చూస్తే" అంటే ప్రతి level లో మనకి **ఆ level లో చివరి (rightmost) node మాత్రమే** కనిపిస్తుంది, మిగతావి దాని వెనుక దాక్కుంటాయి. కాబట్టి problem నిజానికి: **ప్రతి level యొక్క చివరి node ని ఏరు.**

"ప్రతి level" అనగానే **BFS level-by-level** template గుర్తిస్తుంది. Level ని process చేసేటప్పుడు, size sార్లు loop తిరుగుతాం (i = 0..size-1). **చివరి iteration (i === size - 1)** లో ఉన్న node = ఆ level rightmost → దాన్ని answer లో పెట్టు.

(Alternative: DFS లో **కుడి child ని ముందు** visit చేస్తూ, ప్రతి depth కి **మొదటిసారి** చేరిన node ని record చేయడం – అదే పని చేస్తుంది. కానీ BFS-with-size ఇక్కడ అత్యంత సహజం.)

Optimal Approach:

Insight: BFS level order; ప్రతి level లో i === size-1 అయిన node = rightmost → push.

Solution (JavaScript):

```
function rightSideView(root) {
  const res = [];
  if (root === null) return res;

  const queue = [root];
  while (queue.length > 0) {
    const size = queue.length;           // current level nodes
    for (let i = 0; i < size; i++) {
      const node = queue.shift();
      if (i === size - 1) res.push(node.val); // ← level లో చివరిది = rightmost
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
  }
  return res;
}
```

గమనిక: i === size - 1 (సాధారణంగా కుడి node) rightmost ని ఇస్తుంది – ఎందుకంటే మనం ఎప్పుడూ **left ముందు, right తర్వాత** enqueue చేస్తాం, కాబట్టి ఒక level లో చివరిగా dequeue అయ్యేది సహజంగా rightmost. కొన్ని nodes కి కుడి child లేకపోయినా, "level లో చివరిది" ఎప్పుడూ సరైన rightmost అవుతుంది.

Dry Run: root = [1,2,3,null,5,null,4]

```
queue=[1]
Level: size=1. i=0(last): push 1. children → queue=[2,3]    res=[1]
Level: size=2. i=0: node2, children→[5]. i=1(last): push 3, children→[5,4]    res=[1,3]
Level: size=2. i=0: node5(leaf). i=1(last): push 4.        res=[1,3,4]
queue ఖాళీ → return [1,3,4] ✓
```

Complexity:

- **Time:** $O(n)$ — ప్రతి node ఒకసారి (`shift()` $O(n)$ అయితే index-pointer తో $O(n)$ మొత్తం).
- **Space:** $O(w)$ — queue = గరిష్ట level width. Result $O(h)$ (levels సంఖ్య).

గుర్తుంచుకోవాల్సినది:

"ప్రతి level లో ఒక specific node (rightmost/leftmost/first/last)" అంటే BFS-with-size, $i === \text{size}-1$ (లేదా $i === 0$) trick. **Left Side View** = $i === 0$. ఈ నమూనాని levels aggregate అన్నిటికీ మలచవచ్చు (తర్వాతి problem — level averages). DFS variant (కుడి-ముందు, $\text{depth} === \text{res.length}$ అయితే push) కూడా తెలిస్తే bonus.

సాధారణ తప్పులు:

- **size ని loop లోపల చదవడం:** loop మధ్యలో `queue.length` పెరుగుతుంది (children add అవుతూ). **size ని loop ముందే** ఒక్కసారి capture చెయ్యి. లేకపోతే levels కలిసిపోతాయి.
- **rightmost = ఎప్పుడూ node.right అనుకోవడం:** ఒక level లో rightmost node కి కుడి child లేకపోవచ్చు (ఉదా పైన 2 కి 5 అనే కుడి child). "Level లో చివరిగా dequeue అయ్యేది" సరైన నిర్వచనం, `node.right` కాదు.
- **Null root:** ఖాళీ array return — మొదటి line handle చేసింది.

16. Average of Levels in Binary Tree (LeetCode #637) — Easy

సమస్య: ఒక binary tree root ఇస్తారు. ప్రతి level లోని nodes విలువల సగటు (average) ని పైనుండి కిందకి క్రమంలో array గా return చెయ్యాలి.

Constraints: nodes [1, 10^4] ; $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$ (values పెద్దవి అవ్వొచ్చు — sum overflow గురించి ఆలోచించు).

ఉదాహరణ:

```
      3
     / \
    9  20
     / \
    15  7
```

```
Level 0: [3]      → 3
Level 1: [9,20]   → 14.5
Level 2: [15,7]   → 11
Output: [3, 14.5, 11]
```

ఎలా ఆలోచించాలి:

"ప్రతి level లో ఏదో aggregate (ఇక్కడ average)" – నేరుగా **BFS level-by-level** template. Level ని process చేసేటప్పుడు size (ఆ level nodes count) తెలుసు. Loop లో nodes విలువలు sum లో కూడబెట్టు, చివర్లో $sum / size$ = ఆ level average. అంతే – Right Side View లోని అదే అస్థిపంజరం, "చివరి node" బదులు "sum/count".

Optimal Approach:

Insight: BFS; ప్రతి level కి sum కూడబెట్టి $sum / size$ push.

Solution (JavaScript):

```
function averageOfLevels(root) {  
  const res = [];  
  const queue = [root]; // constraints: కనీసం 1 node ఉంటుంది  
  
  while (queue.length > 0) {  
    const size = queue.length;  
    let sum = 0;  
  
    for (let i = 0; i < size; i++) {  
      const node = queue.shift();  
      sum += node.val; // ఈ level విలువలు కూడబెట్టు  
      if (node.left) queue.push(node.left);  
      if (node.right) queue.push(node.right);  
    }  
    res.push(sum / size); // level average  
  }  
  return res;  
}
```

Overflow గమనిక: node values 2^{31} దగ్గర ఉండి, ఒక level లో చాలా nodes ఉంటే, sum పెద్దదవుతుంది.

Java/C++ లో int sum overflow → long / double వాడాలి. JavaScript లో numbers అన్నీ 64-bit doubles – 2^{53} దాకా integers safe. ఈ constraints లో ($\leq 10^4$ nodes $\times \sim 2^{31}$) sum $\sim 2^{45}$, doubles లో safe. కాబట్టి JS లో ప్రత్యేక చర్య అవసరం లేదు, కానీ interview లో "ఏ భాషలో?" బట్టి overflow ప్రస్తావించడం మెచ్చుతారు.

Dry Run: root = [3,9,20,null,null,15,7]

```
queue=[3]  
Level: size=1, sum=3 → avg 3/1=3. children→[9,20]    res=[3]  
Level: size=2, sum=9+20=29 → 29/2=14.5. children→[15,7]  res=[3,14.5]  
Level: size=2, sum=15+7=22 → 22/2=11.                res=[3,14.5,11]  
return [3, 14.5, 11] ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి.
- **Space:** $O(w)$ – queue = గరిష్ట level width. (Result $O(\text{levels}) = O(h)$.)

గుర్తుంచుకోవాల్సినది:

"**ప్రతి level కి aggregate (avg/sum/max/min)**" = BFS-with-size, level లోపల కూడబెట్టి level ముగింపులో push. Right Side View తో సోదర problem — అస్థిపంజరం ఒకటే, లోపలి "పని" మారుతుంది. ఇదే idea తో **Largest Value in Each Row** (#515), **Level Order Sum** లాంటివి. Numeric aggregate లో **overflow / precision** ప్రస్తావించడం SSE మెచ్చుకోలు.

సాధారణ తప్పులు:

- **Integer division:** కొన్ని భాషల్లో `sum / size` integer division అవుతుంది ($14.5 \rightarrow 14$). JS లో `/` float ఇస్తుంది కాబట్టి safe; Java/C++ లో `(double)sum / size`.
- **size capture:** loop లోపల `queue.length` వాడితే children add అవుతూ తప్పు size $\rightarrow$ levels కలుస్తాయి. Loop ముందే capture.
- **sum ని level కి reset చేయకపోవడం:** ప్రతి level మొదట్లో `sum = 0`. మర్చిపోతే averages తప్పు.
- **Overflow (non-JS):** పెద్ద values కి `int` sum overflow. `long / double` వాడు.

17. Binary Tree Level Order Traversal (LeetCode #102) — Medium

సమస్య: ఒక binary tree root ఇస్తారు. **level order traversal** return చెయ్యాలి — అంటే **ప్రతి level ని ఒక వేరే array** గా, పైననుండి కిందకి, ఎడమ నుండి కుడికి. ఫలితం arrays యొక్క array (`number[][]`).

Constraints: nodes `[0, 2000]` ; `-1000 <= Node.val <= 1000`.

ఉదాహరణ:

```

      3
     / \
    9  20
     / \
    15  7
  
```

Output: `[[3], [9,20], [15,7]]`

ఎలా ఆలోచించాలి:

ఇది BFS level pattern యొక్క **canonical (మూలం) problem** — పైన BFS primer template ని అచ్చంగా ఇదే. ప్రతి level ని విడిగా array గా కావాలి, కాబట్టి level-by-level: size snapshot తీసుకో, ఆ size సార్లు dequeue చేస్తూ ఒక level array నింపు, level పూర్తయ్యాక result లో push. Right Side View, Averages — అన్నీ ఈ problem యొక్క variants. ఇక్కడ level మొత్తం భద్రపరుస్తాం.

DFS తో కూడా చేయవచ్చు: depth ని పరామితిగా carry చేస్తూ, `res[depth]` లేకపోతే కొత్త array సృష్టించి, `res[depth].push(node.val)`. కానీ "level order" కి BFS సహజ ఎంపిక.

Optimal Approach:

Insight: BFS level-by-level; ప్రతి level ను ఒక array గా collect చేసి result లో push.

Solution (JavaScript):

```
function levelOrder(root) {
  const res = [];
  if (root === null) return res;          // ఖాళీ చెట్టు → []

  const queue = [root];
  while (queue.length > 0) {
    const size = queue.length;             // ఈ level nodes count
    const level = [];                     // ఈ level విలువలు

    for (let i = 0; i < size; i++) {
      const node = queue.shift();
      level.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    res.push(level);                       // ఒక్క level పూర్తి → జోడించు
  }
  return res;
}
```

Dry Run: root = [3,9,20,null,null,15,7]

```
queue=[3]
size=1: level=[3]. children→[9,20].      res=[[3]]
size=2: level=[9,20]. children→[15,7].    res=[[3],[9,20]]
size=2: level=[15,7]. leaves.             res=[[3],[9,20],[15,7]]
return [[3],[9,20],[15,7]] ✓
```

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి enqueue+dequeue.
- **Space:** $O(w)$ – queue గరిష్ట width; output $O(n)$.

గుర్తుంచుకోవాల్సినది:

ఇది **BFS level template యొక్క పునాది problem** – దీన్ని కంఠస్థం చేస్తే, Right Side View (level చివరిది), Averages (level sum/count), Zigzag (level reverse), Largest per Row (level max), Level Order II (#107 – result reverse) – అన్నీ ఈ ఒక్క skeleton నుండి పుడతాయి. size snapshot అనేదే BFS heart.

సాధారణ తప్పులు:

- **size snapshot మర్చిపోవడం:** అన్ని level problems కి common bug. Loop లో queue.length పెరుగుతూ levels కలుస్తాయి. Loop ముందే `const size = queue.length`.
- **null root:** [] return చెయ్యాలి (levels లేవు). మొదటి line handle.
- **shift() efficiency:** పెద్ద trees కి index-pointer తో $O(1)$ dequeue (primer లో bfsFast). ఇక్కడ $n \leq 2000$ కాబట్టి shift OK.
- **level ని reset చేయకపోవడం:** ప్రతి level కి కొత్త level = []. లేకపోతే అన్నీ ఒకే array లో పడతాయి.

18. Binary Tree Zigzag Level Order Traversal (LeetCode #103) — Medium

సమస్య: ఒక binary tree root ఇస్తారు. **zigzag (spiral) level order** return చెయ్యాలి — level 0 ఎడమ→కుడి, level 1 కుడి→ఎడమ, level 2 మళ్ళీ ఎడమ→కుడి... ఇలా ప్రతి level కి దిశ మారుతూ.

Constraints: nodes [0, 2000] ; -100 <= Node.val <= 100 .

ఉదాహరణ:



Level 0 (L→R): [3]
Level 1 (R→L): [20,9]
Level 2 (L→R): [15,7]
Output: [[3], [20,9], [15,7]]

ఎలా ఆలోచించాలి:

ఇది Level Order (#102) కి చిన్న twist మాత్రమే. Traversal **అలాగే** ఎప్పుడూ ఎడమ→కుడి BFS చేస్తాం (queue క్రమం మార్చం). కేవలం **ఒక్కే level ని result లో పెట్టటప్పుడు దిశ మారుస్తాం**. ఒక boolean leftToRight toggle చేస్తూ: **true** అయితే level ని యథాతథంగా, **false** అయితే **తిరగేసి** (reverse). ప్రతి level తర్వాత flag flip.

Reverse ఎలా? రెండు మార్గాలు: (1) level array కట్టి చివర్లో **.reverse()** . (2) values ని array **ముందు (unshift)** చేప్పించడం — right→left level కి node లు వరుసగా ముందు చేప్పిస్తే తిరగబడతాయి. unshift ఒక్కే call O(k) కావచ్చు; deque లేదా చివర్లో reverse cleaner. కింద unshift version — చదవడానికి సూటిగా.

ముఖ్య హెచ్చరిక: queue క్రమాన్ని మార్చవద్దు (ఎప్పుడూ left ముందు, right తర్వాత enqueue). దిశ మార్పు **output arrangement లో మాత్రమే**, traversal లో కాదు. Queue ని కూడా alternate చేస్తే children క్రమం తారుమారై తప్పు వస్తుంది.

Optimal Approach:

Insight: సాధారణ BFS level order; leftToRight flag తో ప్రతి level ను యథా/reverse గా పేర్చు; ప్రతి level తర్వాత flag toggle.

Solution (JavaScript):

```
function zigzagLevelOrder(root) {
  const res = [];
  if (root === null) return res;

  const queue = [root];
  let leftToRight = true;           // level 0 ఎడమ→కుడి

  while (queue.length > 0) {
```

```

const size = queue.length;
const level = [];

for (let i = 0; i < size; i++) {
  const node = queue.shift();
  // దిశ బట్టి: L→R అయితే వెనుక (push), R→L అయితే ముందు (unshift)
  if (leftToRight) level.push(node.val);
  else level.unshift(node.val);

  // ★ enqueue ఎప్పుడూ ఎడమ ముందు, కుడి తర్వాత (దిశతో సంబంధం లేదు)
  if (node.left) queue.push(node.left);
  if (node.right) queue.push(node.right);
}
res.push(level);
leftToRight = !leftToRight; // తర్వాతి level కి దిశ flip
}
return res;
}

```

Dry Run: root = [3,9,20,null,null,15,7]

```

leftToRight=true
Level: size=1. node3 → push → level=[3]. children→[9,20]. res=[[3]]. flag=false
Level: size=2. node9 → unshift → [9]; node20 → unshift → [20,9].
  children→[15,7]. res=[[3],[20,9]]. flag=true
Level: size=2. node15 → push → [15]; node7 → push → [15,7].
  res=[[3],[20,9],[15,7]]. flag=false
return [[3],[20,9],[15,7]] ✓

```

గమనించు: queue ఎప్పుడూ [9,20], [15,7] క్రమంలోనే ఉంది; కేవలం level 1 ని unshift తో తిరగేశాం.

Complexity:

- **Time:** $O(n)$ – ప్రతి node ఒకసారి. (unshift ఒక్కో level కి $O(\text{width})$; మొత్తం అన్ని levels కలిపి $O(n)$ – ప్రతి node ఒక్కో unshift.)
- **Space:** $O(w)$ – queue width; output $O(n)$.

గుర్తుంచుకోవాల్సినది:

"Traversal మార్చకుండా, output arrangement ని toggle చేయడం" – zigzag కి కీలక insight. Level Order (#102) master అయితే, ఒక leftToRight flag + reverse/unshift జోడిస్తే zigzag. Queue క్రమం స్థిరంగా ఉంచడం (traversal invariant) చాలా BFS variants కి పాఠం. Reverse cost ని తగ్గించాలంటే **deque** (రెండు చివర్ల నుండి) వాడొచ్చు.

సాధారణ తప్పులు:

- **Queue క్రమాన్ని alternate చేయడం (అతిపెద్ద bug):** "R→L level కి కుడి child ముందు enqueue" అని చేస్తే తర్వాతి levels తారుమారు. **Enqueue ఎప్పుడూ ఒకేలా;** దిశ output లో మాత్రమే.
- **flag toggle మర్చిపోవడం:** ప్రతి level తర్వాత leftToRight = !leftToRight . మర్చిపోతే అన్నీ ఒకే దిశ.
- **reverse ని ప్రతిసారి res మొత్తం మీద చేయడం:** కేవలం current level ని reverse; పాత levels ముట్టుకోవద్దు.
- **null root:** [] return – handle అయింది.

Pattern: BST property

వివరణ

BST (Binary Search Tree) అంటే — ఒక binary tree, కానీ ఒక కీలక **నియమం** తో: ప్రతి node కి,

- ఎడమ subtree లోని అన్ని values ఆ node కన్నా చిన్నవి, మరియు
- కుడి subtree లోని అన్ని values ఆ node కన్నా పెద్దవి.

"అన్ని values" అనేదే మెలిక — కేవలం direct children కాదు, **మొత్తం subtree**. ఈ నియమం recursive గా ప్రతి node కి వర్తిస్తుంది. (LeetCode సాధారణంగా duplicates లేని strict BST అనుకుంటుంది.)

ఈ నియమం రెండు మహా-శక్తులు ఇస్తుంది:

- 1. Search / Insert $O(h)$:** ఒక value వెతకాలంటే — root తో పోల్చు, చిన్నదైతే ఎడమకి, పెద్దదైతే కుడికి. ప్రతి అడుగులో సగం చెట్టు తొలగిపోతుంది (balanced అయితే). ఇది array binary search యొక్క tree రూపం — **$O(h)$** , balanced కి $O(\log n)$.
- 2. Inorder = Sorted! (అత్యంత కీలకం):** BST ని **inorder** (left → root → right) traverse చేస్తే, values **పెరుగుతున్న (ascending sorted) క్రమంలో** వస్తాయి! ఎందుకంటే ప్రతి node కి ముందు దాని ఎడమ (చిన్నవి), తర్వాత అది, తర్వాత కుడి (పెద్దవి). **BST + "sorted / kth / closest / min difference / range"** అనే మాట వినగానే → **inorder traversal!** ఈ ఒక్క connection చాలా BST problems ని unlock చేస్తుంది.

Validate కి జాగ్రత్త: BST valid ఆ అని చూడాలంటే, కేవలం `node.left.val < node.val < node.right.val` (direct children) సరిపోదు — **subtree మొత్తం** నియమం పాటించాలి. అందుకే **bounds (low, high) range** ని కిందకి propagate చేస్తూ ప్రతి node ఆ range లో ఉందా చూస్తాం (కింద #98).

Real-life Scenario

ఒక **English dictionary (నిఘంటువు)** ని ఊహించుకో. "M" word వెతకాలంటే, నువ్వు మధ్యలో తెరుస్తావు. "P" కనిపిస్తే — M అంతకన్నా ముందు కదా, ఎడమ సగం వైపు వెళ్ళావు; కుడి సగం మొత్తం వదిలేస్తావు. మళ్ళీ మధ్యలో... ప్రతి అడుగులో సగం పేజీలు తొలగిపోతాయి. అదే BST search — sorted structure వల్ల **సగం-సగం తగ్గించడం**.

Inorder = sorted ని ఇలా అనుకో: నువ్వు BST ని "ఎడమ గోడ నుండి కుడి గోడకి" నిటారుగా చదివితే (ఎడమ subtree పూర్తిగా, తర్వాత node, తర్వాత కుడి), numbers వరుసగా చిన్నవి నుండి పెద్దవి వస్తాయి — ఒక క్రమబద్ధమైన సేన లా.

ఎలా గుర్తించాలి (Recognition Signals)

- **"BST" అని problem లో స్పష్టంగా ఉంటే** — inorder = sorted అనే ఆయుధం మొదట గుర్తుకు తెచ్చుకో.
- **"kth smallest / kth largest"** — inorder లో kth element (#230).
- **"minimum difference / closest value"** — sorted లో adjacent elements మధ్య (#530).
- **"validate / is this a valid BST"** — bounds propagation (#98).
- **"range sum / values between L and R"** — BST లో pruned traversal.
- **"sorted array → balanced BST"** — మధ్య element root (#108).

Template Code (JavaScript)

```
// TreeNode: మిగతా guide లోని అదే { val, left, right }

// 1) BST search - O(h)
function searchBST(root, target) {
  let node = root;
  while (node !== null) {
    if (target === node.val) return node;
    node = target < node.val ? node.left : node.right; // sorted → సగం వదిలేయి
  }
  return null;
}

// 2) Inorder = sorted values (BST కి)
function inorderValues(root, out = []) {
  if (root === null) return out;
  inorderValues(root.left, out);
  out.push(root.val); // ← ఇక్కడ values ascending క్రమంలో వస్తాయి
  inorderValues(root.right, out);
  return out;
}

// 3) Validate - bounds (low, high) propagation (strict)
function isValidBST(root, low = -Infinity, high = Infinity) {
  if (root === null) return true;
  if (root.val <= low || root.val >= high) return false; // range బయట → invalid
  return isValidBST(root.left, low, root.val) // ఎడమకి: పైన బౌండ్ = node
    && isValidBST(root.right, root.val, high); // కుడికి: కింద బౌండ్ = node
}
```

Complexity

- **Search / Insert / Delete: $O(h)$** — balanced $O(\log n)$, skewed $O(n)$.
- **Inorder traversal: $O(n)$** time, $O(h)$ space (recursion/stack).
- **గుర్తుంచుకోవాల్సిన మంత్రం: "BST + sorted-సంబంధ ప్రశ్న = inorder."** ఈ మూడు BST problems (#530, #230, #98) అన్నీ ఈ ఒక్క idea పైనే నిలబడతాయి.

19. Minimum Absolute Difference in BST (LeetCode #530) — Easy

సమస్య: ఒక BST root ఇస్తారు. చెట్టులోని ఏవైనా రెండు వేర్వేరు nodes విలువల మధ్య కనిష్ఠ absolute difference return చెయ్యాలి.

Constraints: nodes $[2, 10^4]$; $0 \leq \text{Node.val} \leq 10^5$. (#783 "Minimum Distance Between BST Nodes" అచ్చం ఇదే problem.)

ఉదాహరణ:



జతల diffs: $|1-2|=1$, $|2-3|=1$, $|3-4|=1$, ... కనిష్ఠం = 1
Output: 1

ఎలా ఆలోచించాలి:

Brute force: అన్ని జతలు (pairs) తీసుకుని diffs లెక్కించడం $O(n^2)$. చెత్త.

కీలక గమనిక: ఇది **BST**! BST లో inorder = **sorted ascending**. Sorted list లో, **కనిష్ఠ difference ఎప్పుడూ ప్రక్కప్రక్క (adjacent) elements మధ్య** ఉంటుంది – దూరంగా ఉన్న elements ఎప్పుడూ ఎక్కువ తేడా. ఉదా sorted $[1, 2, 3, 4, 6]$ లో min diff adjacent జతల్లో (2-1, 3-2, 4-3, 6-4) చూస్తే చాలు, 6-1 లాంటివి అనవసరం.

కాబట్టి plan: **inorder traversal** చేస్తూ, **ముందటి node విలువ (prev)** గుర్తుంచుకో. ప్రస్తుత node దగ్గర **node.val** – prev (sorted కాబట్టి ఇది positive) ని min తో పోల్చు. ప్రత్యేకంగా array కట్టనవసరం లేదు – traversal లోనే running prev, minDiff maintain.

Brute Force / naive:

అన్ని values ఒక array లో collect చేసి, అన్ని జతలు $O(n^2)$, లేదా sort చేసి adjacent ($O(n \log n)$). BST అయితే sort అవసరం లేదు – inorder ఉచితంగా sorted ఇస్తుంది $\rightarrow O(n)$.

Optimal Approach:

Insight: Inorder BST = sorted. Min abs diff = adjacent inorder elements మధ్య min. prev track చేస్తూ ఒకే pass.

Solution (JavaScript):

```

function getMinimumDifference(root) {
  let prev = null;           // inorder లో ముందటి node విలువ
  let minDiff = Infinity;

  function inorder(node) {
    if (node === null) return;

    inorder(node.left);      // 1) ఎడమ (చిన్నవి)

    // 2) node ని process – sorted క్రమంలో వస్తున్నాం
    if (prev !== null) {
      minDiff = Math.min(minDiff, node.val - prev); // adjacent diff (positive)
    }
    prev = node.val;         // ఈ node తర్వాతదానికి "prev" అవుతుంది

    inorder(node.right);     // 3) కుడి (పెద్దవి)
  }

  inorder(root);
}

```

```
return minDiff;
}
```

Dry Run: root = [4,2,6,1,3] (inorder వరుస: 1,2,3,4,6)

```
inorder visits: 1,2,3,4,6 (sorted)
node=1: prev=null → skip. prev=1
node=2: 2-1=1 → minDiff=1. prev=2
node=3: 3-2=1 → minDiff=1. prev=3
node=4: 4-3=1 → minDiff=1. prev=4
node=6: 6-4=2 → min(1,2)=1. prev=6
return 1 ✓
```

Complexity:

- **Time:** $O(n)$ – ఒక్క inorder pass.
- **Space:** $O(h)$ – recursion stack. (Array collect చేయడం $O(n)$ space – ఇది అవసరం లేదు.)

గుర్తుంచుకోవాల్సినది:

"BST + difference/closest = inorder లో adjacent elements." Sorted లో min diff ఎప్పుడూ ప్రక్కప్రక్క – ఈ observation కీలకం. prev ని running గా పట్టుకుని ఒకే pass లో పరిష్కారం (array అవసరం లేదు). ఈ "inorder + prev" నమూనా తర్వాతి problems లోనూ (#98 validate కి కూడా inorder-prev approach ఉంది) వస్తుంది.

సాధారణ తప్పులు:

- **BST ని exploit చేయకపోవడం:** అన్ని జతలు $O(n^2)$ – inorder $O(n)$ ఉండగా వ్యర్థం. "BST" అనే మాట = sorted అనే క్లూ.
- **Math.abs అవసరమా?** Inorder sorted ascending కాబట్టి node.val – prev ఎప్పుడూ ≥ 0 – abs అవసరం లేదు. కానీ ఇది BST అని నమ్మకంగా ఉంటేనే; సందేహం ఉంటే Math.abs safe.
- **prev init null vs first node:** మొదటి node కి prev లేదు → skip. prev = -Infinity పెట్టి skip మర్చిపోతే, తప్పు diff (node.val - (-Infinity)). null check మంచిది.
- **Global array collect + sort:** BST లో అనవసరం; inorder అప్పటికే sorted.

20. Kth Smallest Element in a BST (LeetCode #230) – Medium

సమస్య: ఒక BST root మరియు ఒక సంఖ్య k ఇస్తారు. చెట్టులో k-వ అతిచిన్న (kth smallest) విలువ return చెయ్యాలి (1-indexed – k=1 అంటే అతిచిన్నది).

Constraints: nodes [1, 10⁴] ; 1 ≤ k ≤ nodes . **Follow-up:** BST తరచుగా modify (insert/delete) అవుతూ, kth smallest ని పడేపడే అడిగితే – ఎలా optimize?

ఉదాహరణ:



```
2 4
/
1
```

```
sorted (inorder): 1,2,3,4,5,6
k=3 → Output: 3
```

ఎలా ఆలోచించాలి:

మళ్ళీ అదే మంత్రం: **BST + "kth smallest / sorted order" = inorder!** Inorder BST ని ascending sorted గా ఇస్తుంది. కాబట్టి inorder traversal లో **k-వ element** = answer.

Naive: మొత్తం inorder ని array లో collect చేసి arr[k-1] return — O(n) time, O(n) space. పని చేస్తుంది.

మెరుగు: మొత్తం చెట్టు traverse అవసరం లేదు — **kth element దగ్గరే ఆగొచ్చు**. ఒక counter తో inorder చేస్తూ, k-వ node కి చేరగానే return. Iterative inorder (stack) దీనికి బాగా సరిపోతుంది — BST Iterator (#12) లోని అదే "leftmost spine push, pop, right కి దిగు" నమూనా. ప్రతి pop ఒక inorder element; k సార్లు pop చేశాక ఆ node = answer. సగటున kth చిన్నదైతే మొత్తం చెట్టు తిరగనవసరం లేదు.

Brute Force / naive:

```
function kthSmallestNaive(root, k) {
  const arr = [];
  (function inorder(n){ if(!n) return; inorder(n.left); arr.push(n.val); inorder(n.right); })(
  root);
  return arr[k - 1];
}
```

O(n) time, O(n) space — మొత్తం collect. Early-stop లేదు.

Optimal Approach (iterative inorder, early stop):

Insight: Iterative inorder (stack): leftmost దాకా push, pop = తర్వాతి అతిచిన్నది, k ని తగ్గించు; k === 0 అయినప్పుడు ఆ node = answer. Right subtree కి దిగు.

Solution (JavaScript):

```
function kthSmallest(root, k) {
  const stack = [];
  let cur = root;

  while (cur !== null || stack.length > 0) {
    // ఎడమ spine మొత్తం push (అతిచిన్నది పైకి)
    while (cur !== null) {
      stack.push(cur);
      cur = cur.left;
    }

    cur = stack.pop(); // inorder లో తర్వాతి అతిచిన్నది
    k--; // ఒక element లెక్కించాం
    if (k === 0) return cur.val; // ఇదే kth smallest → ఇక్కడే ఆగు
  }
}
```

```

    cur = cur.right;          // ఈ node కుడి subtree కి దిగు
}

return -1; // k valid అయితే ఇక్కడికి రాదు
}

```

Dry Run: root = [5,3,6,2,4,null,null,1] , k = 3

```

push left spine నుండి 5: [5,3,2,1] (cur null అయ్యేదాకా)
pop 1 → k=2. 1.right=null → cur=null
pop 2 → k=1. 2.right=null → cur=null
pop 3 → k=0 → return 3 ✓ (inorder 1,2,3... మూడోది 3)

```

గమనించు: 4,5,6 ని అస్సలు తాకలేదు – kth (3వ) దగ్గరే ఆగాం.

Complexity:

- **Time: $O(h + k)$** – leftmost దాకా దిగడం $O(h)$, తర్వాత k pops. Worst case (skewed / పెద్ద k) $O(n)$.
- **Space: $O(h)$** – stack లో ఒక path ancestors.

Follow-up (తరచూ modify): BST తరచుగా మారుతూ, పదేపదే kth అడిగితే – ప్రతి node లో **దాని subtree size (nodes count)** ని store చేయి (augmented BST). అప్పుడు root దగ్గర: ఎడమ subtree size = L అయితే – $k \leq L$ అయితే ఎడమకి; $k == L+1$ అయితే root; లేకపోతే కుడికి $k-L-1$ తే. ప్రతి query **$O(h)$** (k pops అవసరం లేదు). Insert/delete లో counts update. ఇది strong SSE answer.

గుర్తుంచుకోవాల్సినది:

"BST + kth = inorder కి kth element." Iterative inorder (stack) తో **early stop** – మొత్తం చెట్టు తిరగనవసరం లేదు. Augmented BST (subtree counts) అనే follow-up తెలిస్తే senior signal. #12 (BST Iterator) తో అదే core algorithm – ఒకసారి పట్టుకుంటే రెండూ ఉచితం.

సాధారణ తప్పులు:

- **k off-by-one: k 1-indexed.** k-- చేసి $k == 0$ దగ్గర ఆగడం సరైనది (kth pop). `arr[k-1]` naive లో కూడా -1 గుర్తుంచుకో.
- **kth వద్ద ఆగకపోవడం:** early return మర్చిపోతే $O(n)$ – correct కానీ follow-up spirit miss.
- **inorder బదులు preorder/BFS:** అవి sorted ఇవ్వవు → తప్పు answer. BST కి **inorder** మాత్రమే sorted.
- **Recursion తో early-stop కష్టం:** recursion లో counter global గా పెట్టి, answer దొరికాక మిగతా calls ని skip చేయాలి (flag) – iterative stack ఇక్కడ clean.

21. Validate Binary Search Tree (LeetCode #98) – Medium

సమస్య: ఒక binary tree root ఇస్తారు. అది **valid BST** ఆ చెప్పాలి. Valid BST నిబంధన: ప్రతి node కి – **ఎడమ subtree లోని అన్ని values ఆ node కన్నా strictly చిన్నవి, కుడి subtree లోని అన్ని values strictly పెద్దవి**, మరియు రెండు subtrees కూడా valid BSTs.

Constraints: nodes $[1, 10^4]$; $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$ (INT_MIN/INT_MAX విలువలు కూడా node లో ఉండొచ్చు – bounds init జాగ్రత్త).

ఉదాహరణ:

```
      2           5
     /\         /\
    1  3       1  4      (5 కి కుడి 4 - 4 < 5, invalid!)
               /\
               3  6
Input1: [2,1,3]      → true
Input2: [5,1,4,null,null,3,6] → false (4 అనేది root 5 యొక్క కుడి subtree లో, కానీ 4 < 5)
```

ఎలా ఆలోచించాలి:

మొదటి తప్పు ఆలోచన: "ప్రతి node కి $\text{left.val} < \text{node.val} < \text{right.val}$ చూస్తే చాలు" – **తప్పు!** Input2 చూడు: 4 అనేది 5.right కింద ఉంది. Locally $3 < 4 < 6$ OK, కానీ 4 అనేది 5 కి కుడి subtree లో ఉంది కాబట్టి $4 > 5$ అవ్వాలి – కానీ $4 < 5$. అంటే BST నియమం **direct children మీద కాదు, మొత్తం ancestor chain మీద** ఆధారపడుతుంది. ప్రతి node కి అనుమతించిన **range (low, high)** ఉంది – ఆ range దాని ancestors నిర్ణయిస్తారు.

Bounds approach: root కి range $(-\infty, +\infty)$. ఎడమకి దిగినప్పుడు – ఆ subtree అంతా node కన్నా చిన్నదవ్వాలి → **upper bound = node.val**. కుడికి దిగినప్పుడు – పెద్దదవ్వాలి → **lower bound = node.val**. ప్రతి node దగ్గర $\text{low} < \text{node.val} < \text{high}$ (strict) చూస్తూ, range ని కిందకి కుంచించుకుంటూ పంపు. ఏ node అయినా తన range బయట ఉంటే invalid.

Alternative (inorder): BST valid $\iff$ inorder **strictly increasing**. Inorder చేస్తూ, ప్రతి value ముందటి (prev) కన్నా strictly పెద్దదా చూడు – కాకపోతే invalid. #530 (min diff) లోని అదే "inorder + prev" నమూనా. రెండూ $O(n)$; bounds approach చాలా మందికి స్పష్టం.

Brute Force / naive (తప్పు version): direct children మాత్రమే పోల్చడం – పైన చూపినట్టు Input2 కి తప్పుగా true ఇస్తుంది. కాబట్టి ఇది brute కూడా కాదు, **తప్పు**. సరైన $O(n)$ approach నేరుగా వాడాలి.

Optimal Approach (bounds propagation):

Insight: ప్రతి node కి allowed (low, high) range ని ancestors నుండి propagate; $\text{low} < \text{val} < \text{high}$ strict; ఎడమకి $\text{high}=\text{val}$, కుడికి $\text{low}=\text{val}$.

Solution (JavaScript):

```
function isValidBST(root) {
  // node విలువ (low, high) OPEN interval లో ఉండాలి (strict)
  function validate(node, low, high) {
    if (node === null) return true; // ఖాళీ → valid

    // strict inequality: సమానం కూడా invalid (BST duplicates లేవు)
    if (node.val <= low || node.val >= high) return false;

    // ఎడమకి: గరిష్ట hద్దు ఈ node; కుడికి: కనిష్ట hద్దు ఈ node
    return validate(node.left, low, node.val)
      && validate(node.right, node.val, high);
  }
}
```

```

}

// Infinity వాడితే INT_MIN/INT_MAX node values కూడా safe
return validate(root, -Infinity, Infinity);
}

```

Dry Run: Input2 [5,1,4,null,null,3,6]

```

validate(5, -∞, +∞): -∞ < 5 < +∞ ✓
├ validate(1, -∞, 5): -∞ < 1 < 5 ✓ → children null → true
└ validate(4, 5, +∞): 4 <= low(5)? అవును → return false ✗
మొత్తం: true && false = false ✓ (4 అనేది 5 కుడివైపు కాబట్టి > 5 అవ్వాలి, కానీ కాలేదు)

```

Valid case [2,1,3] :

```

validate(2, -∞, +∞) ✓ → validate(1, -∞, 2) ✓, validate(3, 2, +∞) ✓ → true ✓

```

Complexity:

- **Time:** $O(n)$ — ప్రతి node ఒకసారి.
- **Space:** $O(h)$ — recursion stack.

గుర్తుంచుకోవాల్సినది:

"BST validity = మొత్తం subtree range, direct children కాదు." Ancestors నిర్ణయించే (low, high) range ని kindకి propagate చేయడం — ఈ **bounds propagation** pattern range-constrained tree problems కి కీలకం. రెండో ఆయుధం: **inorder strictly increasing** $\Leftrightarrow$ valid BST. రెండూ interview లో చెప్పగలిగితే బలం. — [Infinity/+Infinity](#) init వాడటం INT edge values ని safe గా handle చేస్తుంది.

సాధారణ తప్పులు:

- **Direct children మాత్రమే పోల్చడం (అతిపెద్ద క్లాసిక్ trap):** `node.left.val < node.val < node.right.val` సరిపోదు — deep descendant range violation ని miss చేస్తుంది (Input2). **Range propagate** చేయాల్సిందే.
- **strict vs non-strict:** `<` vs `<=`. LeetCode BST duplicates అనుమతించదు → **strict** (`<=`, `>=` తో reject). `<` వాడితే సమాన values invalid గా గుర్తించవు.
- **bounds ని node values తో init చేయడం:** `low = Number.MIN_SAFE_INTEGER` వాడితే, node లో అదే value ఉంటే తప్పు reject. `-Infinity` safe.
- **Inorder approach లో prev init:** `prev = -Infinity` (లేదా null) తో strict `>` check; మొదటి node కి skip. Duplicate detection కి strict comparison.

ముగింపు (Summary)

అభినందనలు! ఈ 21 problems + 3 pattern primers పూర్తి చేశావంటే, tree problems కి కావాల్సిన **మూడు మహా-ఆయుధాలు** నీ చేతిలో ఉన్నాయి:

1. **Tree DFS (recursion):** "ఈ subtree నుండి నాకు ఏమి కావాలి?" – children ని అడిగి, వాళ్ళ answers ని combine చేయడం. Postorder (children → node) చాలా problems కి. Preorder/Inorder ఎప్పుడు అనేది గుర్తుంచుకో.
2. **Tree BFS (queue + size snapshot):** level-by-level. "level / row / right view / average / zigzag" = ఈ template. `size = queue.length` snapshot అనేదే గుండె.
3. **BST property (inorder = sorted, bounds):** "BST + sorted/kth/closest/diff" = inorder. "validate/range" = bounds propagation.

చివరి సలహా: ప్రతి tree problem ముందు నిన్ను నువ్వు అడుగు – "ఇది DFS-postorder ఆ (children నుండి కూడబెట్టాలా)? BFS-level ఆ (అంతస్తులవారీగా కావాలా)? BST-inorder ఆ (sorted order అవసరమా)?" ఈ మూడు ప్రశ్నలు 90% tree problems కి దారి చూపిస్తాయి. Pattern గుర్తుపడితే, code దానంతట అదే వస్తుంది. All the best నీ SSE interview కి!